

Reconnaître un titre à partir de cent événements de carnet d'ordres

Unités naturelles, tokens d'événements, SignatureNet
et correction de prior transductive sous changement de période

Projet CFM — Thomas de Portzamparc
rédigé avec l'assistance de Claude (Anthropic)

6 octobre 2026 — code CFM/ version 0.2.0

Résumé

On cherche à identifier lequel de $K = 24$ titres a produit une fenêtre de $L = 100$ événements consécutifs d'un carnet d'ordres agrégé, lorsque le test provient d'une période différente de celle de l'entraînement. Ce document décrit, équations à l'appui, le pipeline *CFM Signature Lab* et le compare au pipeline précédent (Regime Lab V2, CFM Reaction R1/R2, V_init). Trois idées le structurent : (i) réexprimer la donnée dans ses *unités naturelles*, ticks et lots, plutôt qu'en multiples de médianes locales ; (ii) transformer chaque événement en un *n-uplet de tokens discrets*, aux seuils fixés a priori, consommé par un encodeur convolutif et attentionnel (SignatureNet) ; (iii) sélectionner les modèles sur un protocole qui reconnaît que la validation aléatoire est biaisée par la dépendance intra-journalière, puis corriger le *décalage de prior* induit par le changement de période grâce à un équilibrage de Sinkhorn. Le premier run atteint une accuracy publique de 0,5700, puis 0,5958 après équilibrage, contre 0,5072 pour le meilleur résultat antérieur. Trois itérations plus tard (augmentation de profondeur, entraînement plus long, alignement de profondeur et surtout vote entre voisins), le score atteint **0,6351**. Nous montrons aussi les limites : la validation aléatoire surestime le score public de 28 points, l'apport des différentes briques n'est pas encore séparé causalement, et l'équilibrage est transductif.

Table des matières

1	Introduction	4
1.1	La question	4
1.2	Ce qui a changé, en une page	4
1.3	Contributions et honnêteté du propos	5
2	Le problème et ses données	5
2.1	Notation	5
2.2	Structure hiérarchique : titres, jours, fenêtres	6
2.3	Changement de période	6
3	Avant / maintenant : deux pipelines côte à côte	6
3.1	Comparaison détaillée par étape	6
4	Couche de données : un cache brut validé	9
4.1	Contrats vérifiés à la lecture	9
4.2	Catégories	9
4.3	Identité des ordres : ne garder que les égalités	9
4.4	Estimation et vérification du tick	10
4.5	Disposition du cache	10

5 Unités naturelles : ticks et lots	11
5.1 Les grandeurs de base	11
5.2 Ce que la normalisation par médiane locale efface	11
5.3 Lots : la convention de taille	12
5.4 Une redondance évitée : le microprice	12
5.5 Données invalides	12
6 Parcours d'ordres et tokens d'événements	12
6.1 Parcours d'un ordre dans une fenêtre tronquée	12
6.2 Tokens : discrétiser sans rien apprendre	13
6.3 Canaux continus par événement	14
7 Un exemple complet : de la fenêtre brute aux tokens	14
7.1 Étape 0 : les événements bruts	14
7.2 Étape 1 : l'identité des ordres	15
7.3 Étape 2 : passer en unités naturelles	16
7.4 Étape 3 : discrétiser en tokens	16
7.5 Ce que l'exemple révèle : D_t est mesuré <i>après</i> l'événement	17
7.6 Étape 4 : des tokens au vecteur d'entrée	18
7.7 Étape 5 : les statistiques de la fenêtre	18
7.8 Pourquoi pas une normalisation relative : un contre-exemple chiffré	19
8 Blocs de features de fenêtre	20
8.1 Le registre	20
8.2 Opérateurs	20
8.3 Les onze blocs	20
8.4 Invariances vérifiées	21
9 Protocole : partitions, dépendance intra-journalière, audit	21
9.1 Partitions	21
9.2 Pourquoi la validation aléatoire surestime le test	22
9.3 Ce que mesure chaque partition	22
10 L'entonnoir de screening	22
10.1 Étape 1 : utilité contre dérive, colonne par colonne	23
10.2 Étape 2 : AUC adversariale par bloc	23
10.3 Étape 3 : ajout réentraîné (forward)	23
10.4 Étape 4 : retrait réentraîné (ablation)	24
10.5 Sélection gloutonne	24
11 SignatureNet	24
11.1 Encodage des événements	24
11.2 Blocs convolutifs dilatés	24
11.3 Attention relationnelle « même ordre »	25
11.4 Pooling	25
11.5 Contexte et fusion	25
11.6 Encodeurs de contrôle	26
11.7 Décompte exact des paramètres	26

12 Entraînement, augmentations, refit	26
12.1 Perte	26
12.2 Optimiseur et calendrier	26
12.3 Sélection de l'époque	27
12.4 Augmentations	27
12.5 Refit sur tous les labels	27
12.6 Reproductibilité et reprise	27
12.7 Modèles tabulaires	28
13 Ensemble et correction de prior par Sinkhorn	28
13.1 Ensemble	28
13.2 Le symptôme	28
13.3 Rappel : correction de prior sous <i>label shift</i>	29
13.4 Formulation : projection entropique sur les marges	29
13.5 Algorithme de Sinkhorn–Knopp	29
13.6 Exemple complet	30
13.7 Effet observé sur le test	30
13.8 Conditions de validité	31
14 Round 2 : voisins et indicateurs sans labels	31
14.1 Faire voter les fenêtres d'un même titre-jour	31
14.2 Test préliminaire : l'espace des probabilités ne suffit pas	32
14.3 Indicateurs sans labels sur le test	32
15 Résultats du run 1 (Kaggle, GPU)	33
15.1 Screening univarié	33
15.2 AUC adversariale	34
15.3 Ajout réentraîné	34
15.4 Retrait réentraîné	35
15.5 Modèles	35
15.6 Du stress au leaderboard	36
15.7 Round 2 : une différence à la fois	37
15.8 V3 : validation par grappes, durée, alignement, voisins	37
16 Limites, menaces à la validité, suite	38
16.1 Ce qui n'est pas établi	38
16.2 Prochaines expériences	39
16.3 Conclusion	39
A Notation	39
B Hyperparamètres	39
C Contrats testés (tests/test_lab.py, 15 tests)	40
D Reproduire	40

1 Introduction

1.1 La question

Une fenêtre est une suite de cent événements de carnet : ajouts, mises à jour et suppressions d'ordres, avec l'état des meilleures limites après chaque événement. Le challenge CFM demande de dire de quel titre elle provient. La difficulté n'est pas seulement de classer : il faut le faire *dans une autre période*. Le test n'a pas été tiré au hasard parmi les fenêtres du train. Il provient d'autres jours, où les prix, la liquidité affichée, la répartition des volumes entre places de cotation et la taille des ordres ont pu changer.

Un bon classifieur doit donc reconnaître ce qui fait la *signature* d'un titre : sa grille de prix (le tick), ses conventions de quantité (les lots), la façon dont ses ordres vivent et meurent. Il doit le faire sans s'appuyer sur ce qui ne caractérise qu'un *régime* passager, comme une profondeur particulière à une période donnée.

1.2 Ce qui a changé, en une page

Le tableau 1 résume les différences entre le pipeline précédent et le pipeline présenté ici. La section 3 les détaille avec deux schémas complets.

	Avant (V2, Reaction R1/R2, V_init)	Maintenant (Signature Lab)
Unités	Prix et tailles divisés par des médianes <i>locales</i> à la fenêtre	Ticks pour les prix, lots pour les quantités : des entiers, communs à toutes les fenêtres d'un titre
Événement	Vecteur de 13 à 19 réels standardisés + one-hot	11 tokens discrets (seuils fixés a priori) + 11 canaux continus
Ordres	Répétition, écart depuis l'occurrence précédente	Rang de première apparition, occurrence, action précédente <i>du même ordre</i> , première action ; relation « même ordre » dans l'attention
Encodeur	BiGRU 48 (R2) ou Transformer hybride (V2)	Convolutions dilatées + attention relationnelle + pooling attention/moyenne/max
Features de fenêtre	Moyennes et écarts-types de canaux relatifs	11 blocs nommés, cachés et versionnés (~200 colonnes), passés par un entonnoir de screening
Stress	Queue haute de profondeur, codée en dur	Queue orientée vers le test (basse), choix consigné
Sélection	Validation aléatoire	Stress équilibré ; la validation aléatoire est jugée non prédictive (écart de 28 points)
Sortie	arg max des probabilités	Moyenne de 4 modèles, puis équilibrage de Sinkhorn (transductif)
Score public	0,5072	0,5700 (brut), 0,5958 (équilibré), 0,6351 (V3, avec vote entre voisins)

TABLE 1 – Les différences essentielles.

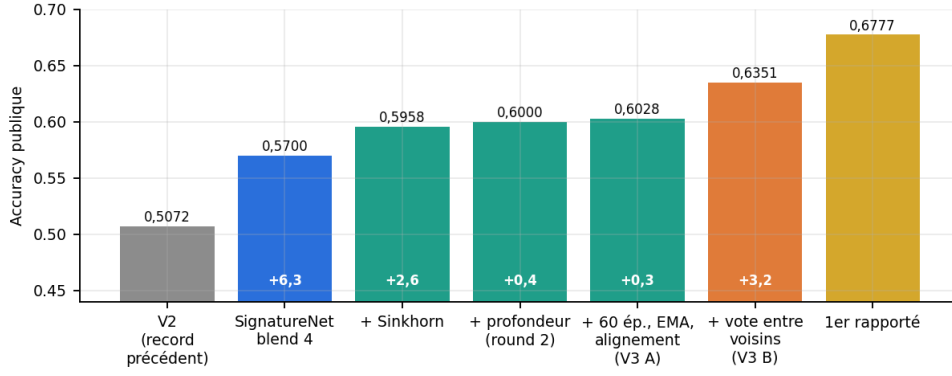


FIGURE 1 – Accuracy publique. Le meilleur classement est celui rapporté par l’auteur à la date de sa consultation. Ce n’est pas une valeur vérifiée ici.

1.3 Contributions et honnêteté du propos

1. Une couche de données *validée* : alignement par `obs_id`, contiguïté, renumérotation des ordres qui ne conserve que les égalités (section 4).
2. Une représentation en unités naturelles. Nous montrons formellement ce que la normalisation par médiane locale efface (section 5).
3. Des tokens d’événements sans aucun seuil appris, et des statistiques de parcours d’ordres calculées de façon vectorisée et exacte (section 6), illustrés sur un exemple complet calculé par le code lui-même (section 7).
4. Un entonnoir de screening : utilité contre dérive, AUC adversariale, ajout et retrait réentraînés, intervalles appariés (section 10).
5. SignatureNet, décrit équation par équation, avec un décompte exact de ses paramètres (section 11).
6. La correction de prior par Sinkhorn : dérivation, convergence, conditions de validité, effet observé (section 13).

Ce que ce document *ne* prétend *pas*. Les +6,3 points du premier run mêlent représentation, architecture et ensemble : un seul score public ne permet pas de les séparer. Les gains de validation ne sont pas des gains de test. L’équilibrage suppose un test équilibré entre titres et utilise les entrées du test : sa conformité au règlement est à vérifier. Enfin, les résultats synthétiques cités en annexe valident le logiciel, jamais la performance.

2 Le problème et ses données

2.1 Notation

Le jeu d’entraînement contient $N_{tr} = 160\,800$ fenêtres, et le test $N_{te} = 81\,600$. Une fenêtre W est une suite ordonnée

$$W = (e_1, \dots, e_L), \quad L = 100, \quad (1)$$

et chaque événement est un n-uplet

$$e_t = (v_t, o_t, a_t, s_t, p_t, b_t, \alpha_t, q_t^b, q_t^a, \tau_t, f_t) \quad (2)$$

où :

- $v_t \in \{0, \dots, 5\}$ est la *venue*, c’est-à-dire la place où l’événement a lieu ;
- o_t est l’identifiant anonymisé de l’ordre, local à la fenêtre ;
- $a_t \in \{A, D, U\}$ est l’action : ajout, suppression ou mise à jour ;

- $s_t \in \{A, B\}$ est le côté : vente (ask) ou achat (bid) ;
- p_t est le prix de l'ordre concerné ;
- b_t et α_t sont le meilleur bid et le meilleur ask du carnet *agrégé entre places*, après l'événement ;
- q_t^b et q_t^a sont les quantités affichées à ces meilleures limites ;
- $\tau_t \in \{0, 1\}$ indique si une suppression ou une mise à jour résulte d'une transaction ;
- f_t est la variation signée de volume associée à l'événement.

La cible est $y \in \{0, \dots, K-1\}$, avec $K = 24$.

Recentrage. Les prix sont fournis *décalés* par le premier bid de la fenêtre : $p_t \leftarrow p_t - b_1$, et de même pour b_t et α_t . Le niveau absolu du prix est donc inconnu. Un offset proche de zéro n'est pas un prix : diviser par lui n'a pas de sens.

Carnet agrégé. $(b_t, \alpha_t, q_t^b, q_t^a)$ décrivent le carnet consolidé entre places. La venue v_t décrit l'événement, pas un carnet séparé. Deux événements consécutifs sur des places différentes s'appliquent donc au même état agrégé.

2.2 Structure hiérarchique : titres, jours, fenêtres

D'après la description du challenge rapportée par l'auteur, il y a 20 fenêtres par titre et par jour. Le train couvre donc

$$D_{\text{tr}} = \frac{160\,800}{24 \times 20} = 335 \text{ jours par titre}, \quad D_{\text{te}} = \frac{81\,600}{24 \times 20} = 170.$$

Cette structure a une conséquence centrale (section 9.2). Écrivons une statistique x d'une fenêtre w du titre c le jour d :

$$x_{c,d,w} = \mu_c + \eta_{c,d} + \varepsilon_{c,d,w}, \quad \eta_{c,d} \sim (0, \sigma_\eta^2), \quad \varepsilon \sim (0, \sigma_\varepsilon^2). \quad (3)$$

μ_c est la signature durable du titre. $\eta_{c,d}$ est un effet de journée : niveau de prix, volatilité, profondeur du jour. ε est la variation propre à la fenêtre. Un classifieur peut reconnaître c grâce à μ_c , ce qui transfère dans une autre période. Il peut aussi le reconnaître grâce à $\eta_{c,d}$, s'il a vu d'autres fenêtres du même jour. Ce second mécanisme ne transfère pas : les jours du test sont nouveaux.

2.3 Changement de période

Les périodes de train et de test diffèrent, et aucune date n'est disponible par observation. Nous distinguons trois formes de décalage, que la suite du document traite séparément :

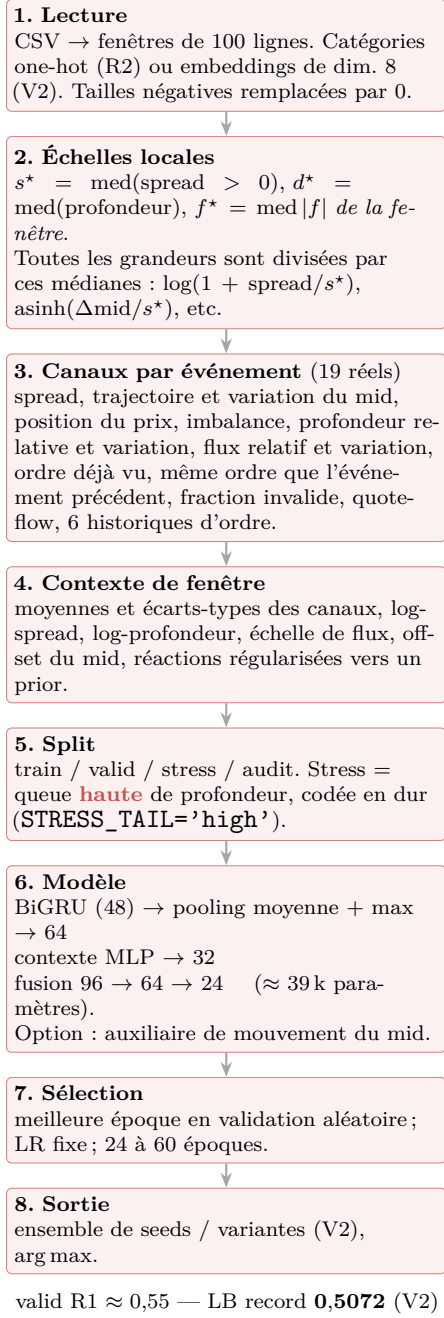
1. **covariate shift** : $p_{\text{te}}(x) \neq p_{\text{tr}}(x)$. Par exemple, les carnets sont moins profonds dans le test ;
2. **décalage de la relation** : $p_{\text{te}}(y | x) \neq p_{\text{tr}}(y | x)$. La même profondeur ne désigne plus les mêmes titres ;
3. **décalage de prior des prédictions** : même si $p_{\text{te}}(y)$ est uniforme, un covariate shift peut rendre la répartition des *prédictions* très non uniforme. C'est ce que corrige la section 13.

3 Avant / maintenant : deux pipelines côte à côte

Cette section sert de carte. Les deux schémas suivent une fenêtre de la lecture du CSV jusqu'à la soumission. Les numéros de section renvoient aux définitions précises.

3.1 Comparaison détaillée par étape

AVANT (Reaction R1/R2, référence V2)



MAINTENANT (Signature Lab)

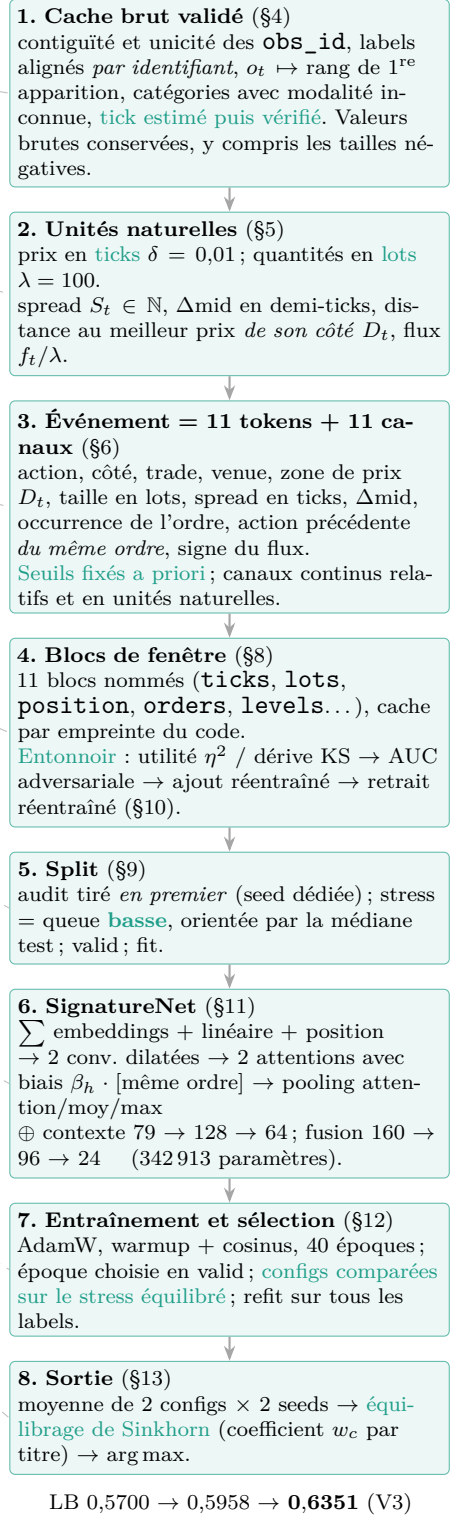


FIGURE 2 – Les deux pipelines, étape par étape. Les pointillés relient les étapes homologues.

Étape	Avant	Maintenant	Pourquoi
Spread	$\log(1 + S/s^*)$, s^* = médiane locale : un spread constant à 1 tick donne $\log 2$, comme un spread constant à 5 ticks	$S_t = (\alpha_t - b_t)/\delta \in \{0, 1, 2, \dots\}$; parts de fenêtre à 1, 2, 3-4, ≥ 5 ticks ; token	Le régime de tick est une signature ($\eta^2 \approx 0,5$, KS $\approx 0,02$, §15)
Prix de l'événement	$\text{asinh}((p - m)/s^*)$, relatif au <i>mid</i>	D_t = distance en ticks au meilleur prix <i>de son côté</i> : dans le spread / au meilleur / 1 / 2 / 3-5 / > 5	Un ordre « 2 ticks derrière le meilleur ask » décrit un comportement, quelle que soit la largeur du spread
Quantités	$\log(1 + q/d^*)$	q/λ (lots), parts de lots ronds / odd lots / mixtes ; token de taille	Les conventions de lot sont propres au titre ; leur dérive est mesurée explicitement
Ordres	« déjà vu », « même que précédent », écart, âge, dernière action	+ rang de 1 ^{re} apparition, première action, <i>action précédente du même ordre</i> , nombre d'événements de l'ordre ; relation « même ordre » dans l'attention	Le cycle de vie $A \rightarrow U \rightarrow D$ d'un ordre est une empreinte, même tronquée
Catégories	one-hot ou embedding dim. 8	tokens additionnés en dim. 96, modalité 0 = inconnu	Toutes les entrées discrètes sont traitées pareil
Tailles < 0	remplacées par 0	conservées brutes ; NaN dans les grandeurs dérivées ; fréquence mesurée	Une convention ne doit pas créer de fausse valeur
Contexte	mean/std de canaux relatifs	11 blocs nommés (~200 colonnes), retenus par l'entonnoir	Hypothèses testables une par une
Stress	queue haute	queue basse, orientée vers le test (transductif, consigné)	Le test est moins profond : médiane $\log(1 + \text{prof.}/\lambda)$ de 1,84 (train) contre 1,70 (test)
Encodeur	BiGRU 48	conv. dilatées + attention relationnelle	Motifs locaux et relations longues séparés
Sélection	valid aléatoire	stress équilibré + indicateurs sans labels	Valid 0,85 contre LB 0,57
Sortie	arg max	Sinkhorn puis arg max	Prédictions biaisées vers les titres peu profonds

TABLE 2 – Comparaison détaillée. s^* , d^* : médianes locales de la fenêtre.

4 Couche de données : un cache brut validé

Cette section décrit `cfm/io.py`. Son principe : aucune transformation *statistique* ici. On lit, on vérifie, on type et on aligne. Ce qui est calculé ensuite dépend de ce cache, et toute incohérence doit y être arrêtée.

4.1 Contrats vérifiés à la lecture

Le CSV est lu par blocs de 20 000 fenêtres ($2 \cdot 10^6$ lignes). Pour chaque bloc de m fenêtres, on remodèle les identifiants en matrice $I \in \mathbb{Z}^{m \times L}$ et on exige :

$$\forall i, \forall t : I_{i,t} = I_{i,1}. \quad (4)$$

Autrement dit, chaque groupe de 100 lignes consécutives porte un seul `obs_id`. Comme la taille de bloc est un multiple de 100 et que le nombre total de lignes l'est aussi (vérifié en première passe), aucune fenêtre ne peut chevaucher deux blocs. On vérifie ensuite l'unicité des `obs_id` sur tout le fichier, puis l'égalité *ensembliste* entre les identifiants de `x_train` et de `y_train`. Les labels sont alignés *par identifiant* :

$$y_i = \mathbf{y_train}[\text{obs_id} = I_{i,1}], \quad (5)$$

et non par position de ligne : un `y_train` mélangé donne le même résultat. C'est l'objet d'un test de contrat.

4.2 Catégories

Les catégories sont normalisées : chaîne, espaces retirés, majuscules, suffixe « .0 » retiré. Elles sont ensuite codées avec des vocabulaires *fixes* pour action, côté et trade : $A \mapsto 1$, $D \mapsto 2$, $U \mapsto 3$; $A \mapsto 1$, $B \mapsto 2$; $\text{FALSE} \mapsto 1$, $\text{TRUE} \mapsto 2$. Le vocabulaire des venues est construit sur le train uniquement, sans labels. Toute valeur hors vocabulaire reçoit le code 0 (« inconnu »), et son nombre d'occurrences par partition est consigné dans `meta.json`.

4.3 Identité des ordres : ne garder que les égalités

Les `order_id` sont des entiers anonymisés, locaux à la fenêtre. La distance numérique entre deux identifiants n'a pas de sens économique établi. Seule l'égalité en a un : « c'est le même ordre ». On remplace donc o_t par son *rang de première apparition*.

Définition 4.1 (Rang de première apparition). Pour une fenêtre $(o_1, \dots, o_L)$, notons $\phi(t) = \min\{u \leq t : o_u = o_t\}$ la première position de l'ordre de l'événement t . On pose

$$\rho(t) = |\{\phi(u) : u \leq t\}| - 1, \quad (6)$$

c'est-à-dire le nombre d'ordres distincts apparus avant celui de t . Les identifiants manquants reçoivent chacun un rang propre : ils ne sont jamais égaux entre eux.

Proposition 4.2 (Invariance par renumérotation). Soit π une bijection quelconque de l'ensemble des identifiants. Alors la suite des rangs calculée sur $(\pi(o_1), \dots, \pi(o_L))$ est identique à celle calculée sur $(o_1, \dots, o_L)$.

Démonstration. ϕ ne dépend que des égalités $o_u = o_t$, et π les préserve puisqu'elle est injective. ρ est une fonction de ϕ seulement. $\square$

Conséquence : aucun composant du modèle ne peut exploiter la valeur des identifiants. Le test de contrat applique des renumérotations aléatoires et vérifie l'égalité exacte.

Calcul vectorisé. Pour un bloc de m fenêtres, on forme une clé entière unique par couple (fenêtre, identifiant) :

$$\kappa_{i,t} = i \cdot 2^{32} + (\tilde{o}_{i,t} + 2^{31}), \quad \tilde{o}_{i,t} = \begin{cases} o_{i,t} & \text{si défini} \\ -1 - t & \text{sinon,} \end{cases} \quad (7)$$

puis `np.unique(return_index, return_inverse)` donne, pour chaque clé distincte, sa première position ϕ . Un tri lexicographique par (fenêtre, ϕ) suivi d'un compteur remis à zéro à chaque nouvelle fenêtre donne ρ . Le coût est $O(mL \log(mL))$, sans boucle Python.

4.4 Estimation et vérification du tick

1. Lire les $2 \cdot 10^6$ premières lignes du train. Calculer les spreads $\alpha_t - b_t$, arrondis à 10^{-6} , strictement positifs.
2. Garder les valeurs dont la fréquence est d'au moins 1 % des spreads positifs. Poser $\hat{\delta}$ = la plus petite d'entre elles.
3. Pour $x \in \{p, b, \alpha\}$, mesurer la part hors grille : $\frac{1}{n} \sum \mathbf{1} \left[|x/\hat{\delta} - \text{round}(x/\hat{\delta})| > 10^{-3} \right]$.
4. `grid_ok` $\iff$ part hors grille < 1 % pour b et α . Sinon, avertir : les blocs en ticks deviennent des approximations.

Algorithme 4.1 – Estimation du tick (`estimate_tick`)

Le recentrage soustrait b_1 , qui est lui-même sur la grille, donc les offsets restent des multiples entiers de δ . C'est ce qui rend les quantités « en ticks » *exactement* entières (modulo l'arrondi flottant), alors qu'une division par une médiane ne l'est pas.

4.5 Disposition du cache

Fichier	Forme, type	Contenu
<code>{split}_num.npy</code>	$(n, 100, 6)$ float32	$p, b, \alpha, q^b, q^a, f$ bruts (rien n'est écrêté)
<code>{split}_cat.npy</code>	$(n, 100, 4)$ int8	venue, action, côté, trade ; 0 = inconnu
<code>{split}_oid.npy</code>	$(n, 100)$ int8	$\rho(t)$
<code>{split}_ids.npy</code>	$(n,)$ int64	<code>obs_id</code> dans l'ordre du fichier
<code>train_y.npy</code>	$(n,)$ int16	indice de classe dans <code>meta['classes']</code>

Le cache est lié à l'empreinte des fichiers sources (chemin, taille). Un cache construit sur d'autres CSV est refusé.

5 Unités naturelles : ticks et lots

5.1 Les grandeurs de base

Soit δ le tick estimé ($\delta = 0,01$ sur le challenge) et λ la taille de lot ($\lambda = 100$, valeur de configuration). On définit, pour chaque événement t d'une fenêtre (fichier `features/common.py`) :

$$\text{cotation valide } \mathcal{Q}_t = \mathbf{1}[b_t, \alpha_t \text{ finis, } \alpha_t \geq b_t], \quad (8)$$

$$\text{spread en ticks } S_t = \frac{\alpha_t - b_t}{\delta} \in \mathbb{N} \quad (\text{NaN si } \neg \mathcal{Q}_t), \quad (9)$$

$$\text{mid } m_t = \frac{1}{2}(b_t + \alpha_t), \quad (10)$$

$$\text{variation du mid en demi-ticks } h_t = \frac{2(m_t - m_{t-1})}{\delta} \in \mathbb{Z}, \quad h_1 = 0, \quad (11)$$

$$\text{distance au meilleur prix de son côté } D_t = \frac{1}{\delta} \begin{cases} b_t - p_t & \text{si } s_t = B, \\ p_t - \alpha_t & \text{si } s_t = A, \end{cases} \quad (12)$$

$$\text{tailles valides } \mathcal{V}_t = \mathbf{1}[q_t^b, q_t^a \text{ finis, } \geq 0], \quad (13)$$

$$\text{profondeur } Q_t = q_t^b + q_t^a \quad (\text{NaN si } \neg \mathcal{V}_t), \quad (14)$$

$$\text{imbalance } I_t = \frac{q_t^b - q_t^a}{Q_t} \in [-1, 1] \quad (\text{NaN si } Q_t = 0), \quad (15)$$

$$\text{flux en lots } F_t = f_t / \lambda. \quad (16)$$

$D_t = 0$ signifie « au meilleur prix » ; $D_t > 0$, « plus profond dans le carnet » ; $D_t < 0$, « dans le spread », c'est-à-dire un ordre qui améliore la cote. Un ordre d'achat placé à $b_t - 2\delta$ et un ordre de vente placé à $\alpha_t + 2\delta$ reçoivent tous deux $D_t = 2$: la grandeur est symétrique entre côtés par construction.

Remarque 5.1 (Pourquoi des demi-ticks). m_t est la moyenne de deux multiples de δ : c'est un multiple de $\delta/2$. Donc h_t est un entier. Un mid qui monte d'un demi-tick correspond exactement à une seule cotation (bid ou ask) qui monte d'un tick.

5.2 Ce que la normalisation par médiane locale efface

Le pipeline précédent utilisait des grandeurs relatives à la fenêtre, comme $r_t = S_t / s^*$ avec $s^* = \text{med}\{S_u : S_u > 0\}$.

Proposition 5.2 (Invariance d'échelle, donc perte d'échelle). *Soit g une fonction quelconque de la suite $(r_t)_t$. Pour tout $c > 0$, remplacer (S_t) par (cS_t) laisse g inchangée.*

Démonstration. $s^*(cS) = c s^*(S)$, donc $r_t(cS) = cS_t / (c s^*) = r_t(S)$. □

Corollaire 5.3. *Une fenêtre au spread constant de 1 tick et une fenêtre au spread constant de 5 ticks ont exactement la même représentation relative ($r_t \equiv 1$). Aucun modèle construit sur r seul ne peut les distinguer.*

Or c'est précisément ce qui distingue un titre à *gros tick* d'un titre à *petit tick*. Pour un titre à gros tick, le tick est grand devant la volatilité intra-journalière : son spread reste presque toujours à 1 tick, et son mid saute par demi-ticks. Pour un titre à petit tick, le spread fluctue sur plusieurs ticks. Le pipeline précédent gardait ce niveau dans une feature séparée (log du spread médian). Mais il perdait la *forme* discrète de la distribution : la part exacte du temps à 1 tick, à 2 ticks, etc.

Les parts en ticks conservent cette information :

$$\pi_k(W) = \frac{\sum_t \mathbf{1}[S_t = k]}{\sum_t \mathcal{Q}_t}, \quad k \in \{0, 1, 2\}, \quad \pi_{3-4}, \pi_{\geq 5}. \quad (17)$$

Empiriquement (section 15), $\pi_{\geq 5}$ et π_1 ont un pouvoir séparateur $\eta^2 = 0,56$ et $0,50$, pour une dérive train/test de seulement $KS = 0,02$. C'est la meilleure combinaison utilité/stabilité du screening.

5.3 Lots : la convention de taille

Pour une quantité non nulle $a = |F_t|$, on distingue :

$$\text{odd lot : } a < 1, \quad \text{lot rond : } a \in \{1, 2, \dots\}, \quad \text{mixte : } a > 1, a \notin \mathbb{N}. \quad (18)$$

Les parts de chaque catégorie par fenêtre forment le bloc `lots`. Le pipeline précédent divisait le flux par sa médiane absolue f^* . Or f^* vaut presque toujours exactement λ : sur la figure 04 du pipeline précédent, $\log(1 + f^*)$ est concentré à $\log 101 \approx 4,62$. Cette normalisation revenait donc à diviser par λ , sans exprimer ce qui distingue vraiment les titres : la *fréquence* des odd lots et des tailles mixtes.

5.4 Une redondance évitée : le microprice

Lemme 5.4. Soit le microprice $\mu_t = \frac{\alpha_t q_t^b + b_t q_t^a}{q_t^b + q_t^a}$. Alors $\frac{\mu_t - m_t}{\alpha_t - b_t} = \frac{I_t}{2}$.

$$\text{Démonstration. } \mu_t - m_t = \frac{\alpha_t q^b + b_t q^a}{Q} - \frac{(\alpha_t + b_t)(q^b + q^a)}{2Q} = \frac{(\alpha_t - b_t)(q^b - q^a)}{2Q}. \quad \square$$

Ajouter le microprice normalisé en plus de l'imbalance n'apporterait qu'une copie linéaire.

5.5 Données invalides

Une cotation croisée ($\alpha_t < b_t$), une taille négative ou une valeur non finie produisent des NaN dans les grandeurs dérivées. Elles ne produisent jamais un 0, qui serait une fausse valeur plausible. Les statistiques de fenêtre sont calculées sur les valeurs finies seulement :

$$\bar{x}(W) = \frac{\sum_t x_t \mathbf{1}[x_t \text{ fini}]}{\sum_t \mathbf{1}[x_t \text{ fini}]}, \quad (19)$$

et valent NaN si aucune valeur n'est finie. Les arbres gèrent nativement les NaN. Les réseaux les remplacent par la moyenne du fit (soit 0 après standardisation). La part de tailles nulles à une cotation valide et la part de tailles négatives sont elles-mêmes des features (bloc `lots`). Cette question avait été soulevée sur la figure 01 du pipeline précédent, avec un `ask_size` nul pendant 48 événements.

6 Parcours d'ordres et tokens d'événements

6.1 Parcours d'un ordre dans une fenêtre tronquée

Pour l'événement t , notons $\mathcal{P}(t) = \{u < t : \rho(u) = \rho(t)\}$ les occurrences *antérieures* du même ordre dans la fenêtre. On définit :

$$\text{occurrence } n_t = |\mathcal{P}(t)| \quad (0 = \text{première fois vu}), \quad (20)$$

$$\text{occurrence précédente } \text{prev}(t) = \max \mathcal{P}(t) \quad (\text{indéfini si } n_t = 0), \quad (21)$$

$$\text{écart } g_t = t - \text{prev}(t) \quad (0 \text{ si } n_t = 0), \quad (22)$$

$$\text{action précédente du même ordre } a_t^{\leftarrow} = a_{\text{prev}(t)} \quad (0 \text{ si } n_t = 0), \quad (23)$$

$$\text{première action } a_t^{\circ} = a_{\min(\mathcal{P}(t) \cup \{t\})}, \quad (24)$$

$$\text{nombre d'événements de l'ordre } N_t = |\{u \leq L : \rho(u) = \rho(t)\}|. \quad (25)$$

Remarque 6.1 (Troncature). La fenêtre est un extrait. Une première apparition sous forme U ou D révèle un ordre créé *avant* la fenêtre. Une première apparition A est un ajout *observé*. Aucune de ces grandeurs n'est une durée de vie, un volume initial ou une identité de participant. g_t est compté en événements, pas en secondes : aucun horodatage n'est disponible.

Calcul exact et vectorisé. On numérote les événements d'un bloc de m fenêtres à plat, $j = iL + t$, et on forme la clé $k_j = 128i + \rho_{i,t}$, avec $\rho < 100 < 128$. Un tri *stable* σ des clés regroupe les événements de chaque (fenêtre, ordre). Comme le tri est stable et que les positions plates sont croissantes dans le temps, les occurrences d'un même ordre restent dans l'ordre chronologique. Notons $\kappa = k_{\sigma(\cdot)}$ la suite triée et $\nu_r = \mathbf{1}[r = 0 \text{ ou } \kappa_r \neq \kappa_{r-1}]$ le début de groupe. Alors

$$\text{début}(r) = \max\{r' \leq r : \nu_{r'} = 1\}, \quad (26)$$

$$n_{\sigma(r)} = r - \text{début}(r), \quad (27)$$

$$\text{prev}(\sigma(r)) = \begin{cases} \sigma(r-1) & \text{si } \nu_r = 0, \\ \text{indéfini} & \text{sinon,} \end{cases} \quad (28)$$

$$a_{\sigma(r)}^\circ = a_{\sigma(\text{début}(r))}. \quad (29)$$

Proposition 6.2. *Ces formules donnent exactement les définitions ci-dessus.*

Démonstration. Dans un groupe (même fenêtre, même ρ), le tri stable conserve l'ordre croissant des positions plates, donc l'ordre chronologique. Le r -ième élément d'un groupe a donc $r - \text{début}(r)$ prédécesseurs dans ce groupe ; son prédécesseur immédiat est l'élément trié juste avant lui ; et le premier élément du groupe est la première apparition. $\square$

Le test de contrat compare ces calculs à une boucle naïve sur des données aléatoires.

6.2 Tokens : discrétiser sans rien apprendre

Définition 6.3 (Bucket à seuils fixés). Pour des seuils $c_1 < \dots < c_J$ choisis *a priori*, on pose

$$B(x; c) = \begin{cases} 0 & x \text{ non fini,} \\ 1 + \#\{j : c_j \leq x\} & \text{sinon,} \end{cases} \quad B \in \{0, \dots, J+1\}. \quad (30)$$

Token	Grandeur	Card.	Valeurs
tok_action	a_t	4	inconnu, A, D, U
tok_side	s_t	3	inconnu, A, B
tok_trade	τ_t	3	inconnu, non, oui
tok_venue	v_t	16	inconnue, 6 places (réservé jusqu'à 15)
tok_pos	$B(D_t; -.5, .5, 1.5, 2.5, 5.5)$	7	?, dans le spread, au meilleur, 1, 2, 3-5, > 5 ticks
tok_size	$B(F_t ; 10^{-9}, 1^\pm, 2^+, 5^+, 10^+)$	8	?, 0, odd, = 1 lot, (1, 2], (2, 5], (5, 10], > 10
tok_spread	$B(S_t; .5, 1.5, 2.5, 4.5)$	6	?, 0, 1, 2, 3-4, ≥ 5 ticks
tok_dmid	$B(h_t; -1.5, -.25, .25, 1.5)$	6	?, ≤ -1.5 , baisse, 0, hausse, ≥ 1.5 demi-tick
tok_occ	$1 + \min(n_t, 3)$	5	1 ^{re} , 2 ^e , 3 ^e , 4 ^e + occurrence
tok_prev_action	$a_t^\leftarrow$	4	aucune, A, D, U
tok_fluxsign	$B(f_t; -10^{-12}, 10^{-12})$	4	?, < 0, 0, > 0

TABLE 3 – Les onze tokens. $1^\pm$ désigne les deux seuils 1 ∓ 10^{-9} qui isolent « exactement un lot ».

Proposition 6.4 (Absence de fuite par construction). *L'application $e_t \mapsto (\text{tokens})$ est une fonction déterministe de l'événement, de sa fenêtre et des constantes (δ, λ) . Elle ne dépend d'aucune autre fenêtre ni d'aucun label. En particulier, elle est identique au développement, au refit et sur le test.*

Le seul paramètre « appris » en amont est $\hat{\delta}$. Il est estimé sur les prix du train, sans labels, et il est vérifié égal à la grille (0,01).

Taille du vocabulaire. Le produit cartésien des cardinalités vaut

$$4 \cdot 3 \cdot 3 \cdot 16 \cdot 7 \cdot 8 \cdot 6 \cdot 6 \cdot 5 \cdot 4 \cdot 4 = 92\,897\,280 \approx 9,3 \cdot 10^7, \quad (31)$$

mais un événement n'est pas représenté par un mot de ce produit. Il est représenté par la *somme* de 11 embeddings (section 11.1), qui ne coûte que $\sum_j \text{card}_j = 66$ vecteurs. Les interactions entre tokens (« une suppression, côté ask, au meilleur ») sont apprises par les couches suivantes.

6.3 Canaux continus par événement

En complément des tokens, chaque événement porte des canaux réels :

$$\text{ev_relative} : (I_t, \log(1 + Q_t/d^*), \text{asinh}(\Delta Q_t/d^*), \text{slog}(f_t/f^*), \log(1 + g_t)), \quad (32)$$

$$\text{ev_levels} : (\log(1 + q_t^b/\lambda), \log(1 + q_t^a/\lambda), \text{slog } F_t, \log(1 + S_t), \text{asinh } D_t, \text{asinh } h_t), \quad (33)$$

avec $\text{slog}(x) = \text{sign}(x) \log(1 + |x|)$, et d^*, f^* les médianes positives de la fenêtre (repli à 1). La version 0.2.0 sépare `ev_levels` en `ev_ticks` (les trois derniers canaux) et `ev_sizes` (les trois premiers), pour pouvoir retirer les tailles absolues sans retirer les ticks.

7 Un exemple complet : de la fenêtre brute aux tokens

Les définitions des sections 5 et 6 sont plus faciles à saisir sur un cas réel. Nous construisons à la main un motif de 20 événements qui passe par les situations intéressantes :

- un ordre déjà présent avant la fenêtre ;
- des ajouts au meilleur prix et plus profond ;
- des exécutions, dont une qui vide le meilleur niveau ;
- un odd lot, une taille mixte ;
- une amélioration de la cote, puis une annulation immédiate ;
- un même ordre qui change de venue ;
- un gros ordre lointain.

La fenêtre passée au code répète ce motif cinq fois (100 événements, avec des identifiants renouvelés à chaque répétition).

Tous les nombres de cette section sont produits par le code du pipeline (`cfm.features`), via le script `paper/examples/make_examples.py`. Ils ne sont pas recopiés à la main.

7.1 Étape 0 : les événements bruts

Prix recentrés sur le premier bid, $\delta = 0,01$ et $\lambda = 100$: c'est exactement ce que contient le CSV du challenge.

t	venue	order_id	act.	côté	price	bid	ask	q^b	q^a	trade	flux
1	0	5012	U	B	0	0	0,01	300	200	non	-100
2	1	7731	A	A	0,01	0	0,01	300	300	non	100
3	1	2210	A	B	-0,02	0	0,01	300	300	non	200
4	0	7731	D	A	0,01	0	0,01	300	200	oui	-100
5	2	9002	A	A	0,02	0	0,01	300	200	non	37
6	0	5012	D	B	0	0	0,01	100	200	non	-200
7	3	1180	D	A	0,01	0	0,02	100	37	oui	-200
8	1	2210	U	B	-0,02	0	0,02	100	37	non	100
9	2	4040	A	A	0,01	0	0,01	100	100	non	100
10	2	4040	D	A	0,01	0	0,02	100	37	non	-100
11	0	3333	A	B	0,01	0,01	0,02	150	37	non	150
12	4	9002	U	A	0,02	0,01	0,02	150	100	non	63
13	4	3333	D	B	0,01	0	0,02	100	100	oui	-150
14	5	8888	A	B	-0,06	0	0,02	100	100	non	1000
15	5	8888	U	B	-0,06	0	0,02	100	100	non	-500
16	0	1212	A	A	0,02	0	0,02	100	200	non	100
17	0	1212	U	A	0,02	0	0,02	100	300	non	100
18	3	1212	D	A	0,02	0	0,03	100	400	oui	-300
19	1	7070	A	A	0,02	0	0,02	100	100	non	100
20	1	2210	D	B	-0,02	0	0,02	100	100	non	-300

TABLE 4 – Le motif brut. B = côté acheteur (bid), A = côté vendeur (ask). Les colonnes bid, ask, q^b et q^a décrivent le carnet agrégé *après* l'événement.

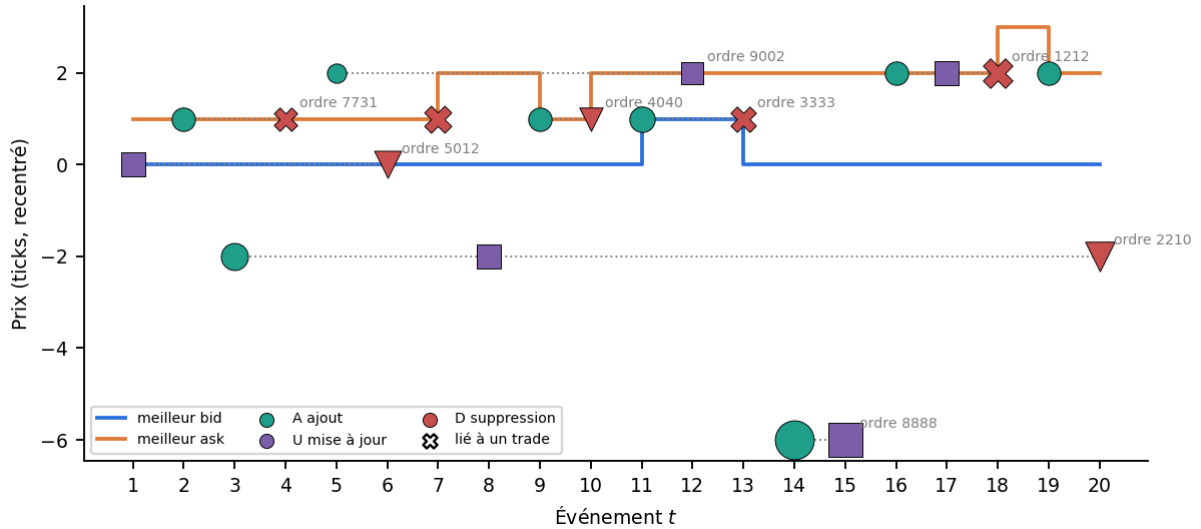


FIGURE 3 – Le motif en image. Les escaliers sont les meilleures limites ; chaque marqueur est un événement (couleur = action, croix = lié à un trade, taille $\propto \sqrt{|f|}$). Les pointillés relient les événements d'un *même ordre* : ce sont les parcours que les tokens d'occurrence et d'action précédente encodent.

7.2 Étape 1 : l'identité des ordres

Les identifiants bruts (5012, 7731, ...) sont remplacés par leur rang de première apparition ρ_t (section 4). Seules les égalités comptent : 5012 devient 0, 7731 devient 1, 2210 devient 2, etc. On en déduit l'occurrence n_t , l'action précédente du même ordre $a_t^{\leftarrow}$ et l'écart g_t (tableau 5). Trois parcours se lisent directement :

- **L'ordre 2210** ($\rho = 2$) apparaît à $t = 3$ (ajout), $t = 8$ (mise à jour) et $t = 20$ (suppression). On obtient $n = (0, 1, 2)$, $a^{\leftarrow} = (-, A, U)$ et $g = (0, 5, 12)$.
- **L'ordre 5012** ($\rho = 0$) apparaît d'abord sous forme d'une *mise à jour*. Il existait donc avant la fenêtre : la troncature est visible. Dans le bloc `orders`, il compte dans `first_seen_U`.

- **L'ordre 1212** ($\rho = 8$) est ajouté puis modifié sur la venue 0, puis exécuté sur la venue 3 ($t = 18$). C'est une répétition « inter-venue », possible parce que le carnet est agrégé (`cross_venue_repeat`).

7.3 Étape 2 : passer en unités naturelles

t	ρ_t	n_t	$a_t^{\leftarrow}$	g_t	S_t	h_t	D_t	F_t	lecture
1	0	0	—	0	1	0	0	-1	ordre déjà présent avant la fenêtre (1re apparition = U), réduit d'un lot
2	1	0	—	0	1	0	0	1	ajout d'un lot au meilleur ask
3	2	0	—	0	1	0	2	2	ajout de 2 lots, 2 ticks derrière le meilleur bid
4	1	1	A	2	1	0	0	-1	l'ordre de $t=2$ est exécuté (suppression liée à un trade)
5	3	0	—	0	1	0	1	0,37	odd lot (37 actions), 1 tick derrière le meilleur ask
6	0	1	U	5	1	0	0	-2	annulation de l'ordre pré-existant
7	4	0	—	0	2	1	-1	-2	trade vide le meilleur ask : l'ask passe à 0,02, spread 2 ticks, le mid monte d'un demi-tick
8	2	1	A	5	2	0	2	1	l'ordre profond de $t=3$ grossit d'un lot
9	5	0	—	0	1	-1	0	1	ajout DANS le spread : améliore l'ask
10	5	1	A	1	2	1	-1	-1	le même ordre est annulé aussitôt (ajouté puis supprimé)
11	6	0	—	0	1	1	0	1,5	ajout DANS le spread côté bid, taille mixte (1,5 lot)
12	3	1	A	7	1	0	0	0,63	l'odd lot de $t=5$ est complété à un lot rond
13	6	1	A	2	2	-1	-1	-1,5	l'ordre de $t=11$ est exécuté : le bid redescend
14	7	0	—	0	2	0	6	10	gros ordre (10 lots) très loin (6 ticks)
15	7	1	A	1	2	0	6	-5	réduit de moitié
16	8	0	—	0	2	0	0	1	ajout d'un lot au meilleur ask
17	8	1	A	1	2	0	0	1	le même ordre grossit
18	8	2	U	1	3	1	-1	-3	exécuté sur une AUTRE venue (3) : l'ask passe à 0,03, spread 3 ticks
19	9	0	—	0	2	-1	0	1	ajout dans le spread, l'ask revient à 0,02
20	2	2	U	12	2	0	2	-3	annulation de l'ordre profond de $t=3$

TABLE 5 – Grandeurs dérivées, calculées par `features/common.py` : spread S_t en ticks, variation du mid h_t en demi-ticks, distance au meilleur prix de son côté D_t en ticks, flux F_t en lots.

Quatre calculs détaillés.

- $t = 5$ (odd lot, côté ask, prix 0,02, ask après l'événement 0,01) : $D_5 = (p_5 - \alpha_5)/\delta = (0,02 - 0,01)/0,01 = 1$, soit un tick derrière le meilleur ask ; $F_5 = 37/100 = 0,37 < 1$, donc un odd lot.
- $t = 7$ (un trade vide le meilleur ask) : l'ask passe de 0,01 à 0,02, donc $S_7 = 2$. Le mid passe de 0,005 à 0,010, donc $h_7 = 2 \times 0,005/0,01 = 1$: une hausse d'un demi-tick.
- $t = 11$ (ajout côté bid à 0,01, qui améliore le bid) : après l'événement, $b_{11} = 0,01$, donc $D_{11} = (b_{11} - p_{11})/\delta = 0$, et $F_{11} = 1,5$, une taille mixte.
- $t = 14$ (gros ordre lointain) : $D_{14} = (0 - (-0,06))/0,01 = 6$ et $F_{14} = 10$ lots.

7.4 Étape 3 : discrétiser en tokens

Chaque grandeur passe par une fonction de bucket à seuils fixés, $B(x; c) = 1 + \#\{j : c_j \leq x\}$ (0 si x n'est pas défini).

Pour la taille, les seuils sont $c = (10^{-9}, 1 - 10^{-9}, 1 + 10^{-9}, 2 + 10^{-9}, 5 + 10^{-9}, 10 + 10^{-9})$:

$$t = 5 : B(0,37; c) = 1 + \#\{10^{-9}\} = 2 \Rightarrow \text{« odd »},$$

$$t = 1 : B(1; c) = 1 + \#\{10^{-9}, 1 - 10^{-9}\} = 3 \Rightarrow \text{« = 1 lot »},$$

$$t = 14 : B(10; c) = 1 + 5 = 6 \Rightarrow \text{« (5,10] »}.$$

Les deux seuils 1 ± 10^{-9} isolent exactement « un lot rond », qui est la taille la plus fréquente sur ce marché.

t	act	côté	trade	venue	pos	taille	spread	Δ mid	occ	préc.	signe
1	3 (U)	2 (bid)	1 (non)	1 (v0)	2 (=)	3 (=1 lot)	2 (1)	3 (0)	1 (1re)	0 (aucune)	1 (<0)
2	1 (A)	1 (ask)	1 (non)	2 (v1)	2 (=)	3 (=1 lot)	2 (1)	3 (0)	1 (1re)	0 (aucune)	3 (>0)
3	1 (A)	2 (bid)	1 (non)	2 (v1)	4 (+2)	4 ((1,2])	2 (1)	3 (0)	1 (1re)	0 (aucune)	3 (>0)
4	2 (D)	1 (ask)	2 (oui)	1 (v0)	2 (=)	3 (=1 lot)	2 (1)	3 (0)	2 (2e)	1 (A)	1 (<0)
5	1 (A)	1 (ask)	1 (non)	3 (v2)	3 (+1)	2 (odd)	2 (1)	3 (0)	1 (1re)	0 (aucune)	3 (>0)
6	2 (D)	2 (bid)	1 (non)	1 (v0)	2 (=)	4 ((1,2])	2 (1)	3 (0)	2 (2e)	3 (U)	1 (<0)
7	2 (D)	1 (ask)	2 (oui)	4 (v3)	1 (in)	4 ((1,2])	3 (2)	4 (hausse)	1 (1re)	0 (aucune)	1 (<0)
8	3 (U)	2 (bid)	1 (non)	2 (v1)	4 (+2)	3 (=1 lot)	3 (2)	3 (0)	2 (2e)	1 (A)	3 (>0)
9	1 (A)	1 (ask)	1 (non)	3 (v2)	2 (=)	3 (=1 lot)	2 (1)	2 (baisse)	1 (1re)	0 (aucune)	3 (>0)
10	2 (D)	1 (ask)	1 (non)	3 (v2)	1 (in)	3 (=1 lot)	3 (2)	4 (hausse)	2 (2e)	1 (A)	1 (<0)
11	1 (A)	2 (bid)	1 (non)	1 (v0)	2 (=)	4 ((1,2])	2 (1)	4 (hausse)	1 (1re)	0 (aucune)	3 (>0)
12	3 (U)	1 (ask)	1 (non)	5 (v4)	2 (=)	2 (odd)	2 (1)	3 (0)	2 (2e)	1 (A)	3 (>0)
13	2 (D)	2 (bid)	2 (oui)	5 (v4)	1 (in)	4 ((1,2])	3 (2)	2 (baisse)	2 (2e)	1 (A)	1 (<0)
14	1 (A)	2 (bid)	1 (non)	6 (v5)	6 (>+5)	6 ((5,10])	3 (2)	3 (0)	1 (1re)	0 (aucune)	3 (>0)
15	3 (U)	2 (bid)	1 (non)	6 (v5)	6 (>+5)	5 ((2,5])	3 (2)	3 (0)	2 (2e)	1 (A)	1 (<0)
16	1 (A)	1 (ask)	1 (non)	1 (v0)	2 (=)	3 (=1 lot)	3 (2)	3 (0)	1 (1re)	0 (aucune)	3 (>0)
17	3 (U)	1 (ask)	1 (non)	1 (v0)	2 (=)	3 (=1 lot)	3 (2)	3 (0)	2 (2e)	1 (A)	3 (>0)
18	2 (D)	1 (ask)	2 (oui)	4 (v3)	1 (in)	5 ((2,5])	4 (3-4)	4 (hausse)	3 (3e)	3 (U)	1 (<0)
19	1 (A)	1 (ask)	1 (non)	2 (v1)	2 (=)	3 (=1 lot)	3 (2)	2 (baisse)	1 (1re)	0 (aucune)	3 (>0)
20	2 (D)	2 (bid)	1 (non)	2 (v1)	4 (+2)	5 ((2,5])	3 (2)	3 (0)	3 (3e)	3 (U)	1 (<0)

TABLE 6 – Les onze tokens de chaque événement : l’indice d’abord, la signification entre parenthèses. Pour pos, « in » signifie « dans le spread », « = » « au meilleur » et « + k » « k ticks derrière ».

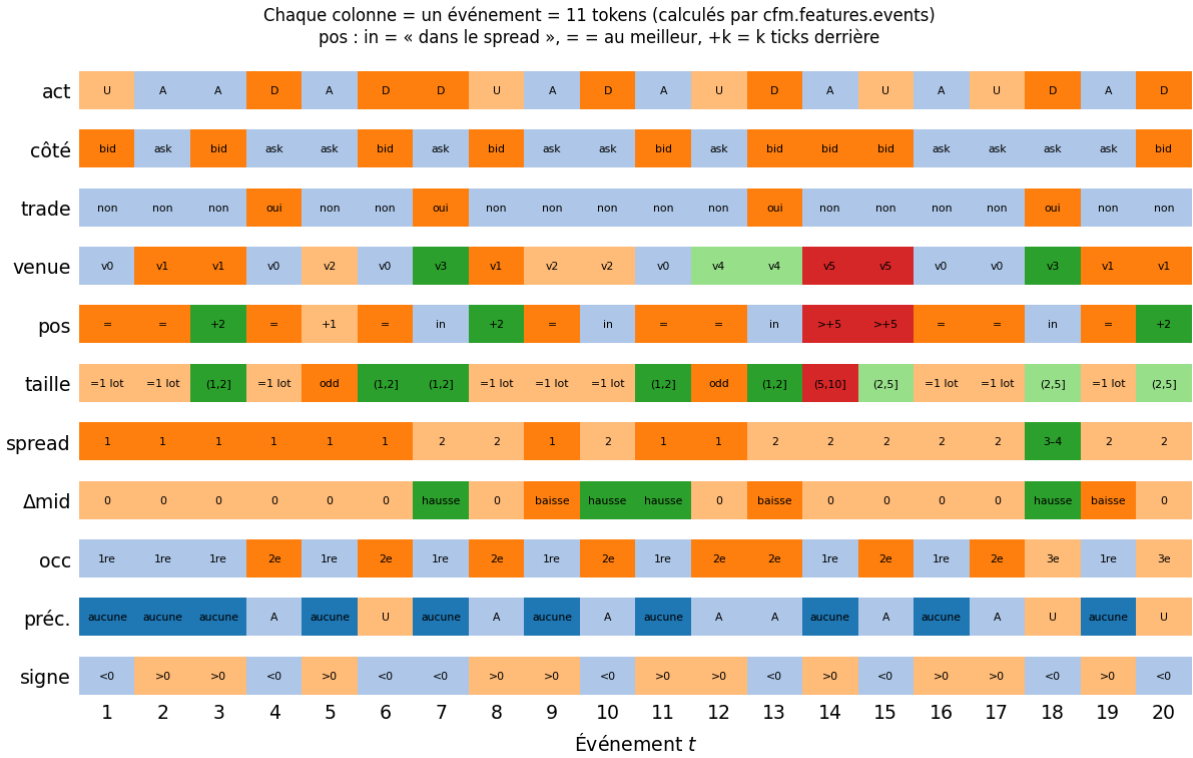


FIGURE 4 – La même information en bandeau : une colonne par événement, une ligne par token. C’est littéralement ce que voit SignatureNet avant ses couches d’embedding.

7.5 Ce que l’exemple révèle : D_t est mesuré *après* l’événement

L’exemple met au jour une subtilité de la définition. Elle n’apparaît pas à la lecture des formules.

- **Ajouts qui améliorent la cote** ($t = 9$, $t = 11$). L’ordre devient lui-même le meilleur prix. Il est donc codé « = » (au meilleur), et non « in » (dans le spread).

- **Suppressions qui vident le meilleur niveau** ($t = 7, 10, 13, 18$). Après l'événement, la meilleure limite a reculé, et le prix de l'ordre supprimé se retrouve à *l'intérieur* du nouveau spread : $D_t = -1$, codé « in ».

Dans le pipeline actuel, le token « in » signifie donc en pratique « *cet événement a vidé le meilleur niveau* ». C'est une information utile, que le modèle a exploitée telle quelle (LB 0,6351), mais son nom est trompeur. La correction naturelle consiste à mesurer la position par rapport au carnet *avant* l'événement :

$$D_t^{\text{pré}} = \frac{1}{\delta} \begin{cases} b_{t-1} - p_t & s_t = B, \\ p_t - \alpha_{t-1} & s_t = A. \end{cases} \quad (34)$$

Avec cette définition, $D_9^{\text{pré}} = (0,01 - 0,02)/0,01 = -1$ (amélioration de la cote : « in »), et $D_7^{\text{pré}} = (0,01 - 0,01)/0,01 = 0$ (suppression au meilleur : « = »).

Les deux versions portent des informations différentes. Garder les deux tokens (« où était l'ordre » et « ce que l'événement a fait au carnet ») est une expérience prévue pour la V5. Elle n'a pas encore été mesurée.

7.6 Étape 4 : des tokens au vecteur d'entrée

L'événement $t = 5$ a les indices de tokens (1, 1, 1, 3, 3, 2, 2, 3, 1, 0, 3) (tableau 6). Son vecteur d'entrée est

$$x_5^{(0)} = \text{LN} (P_5 + E_{\text{act}}[1] + E_{\text{côté}}[1] + E_{\text{trade}}[1] + E_{\text{venue}}[3] + E_{\text{pos}}[3] + E_{\text{taille}}[2] \\ + E_{\text{spread}}[2] + E_{\Delta\text{mid}}[3] + E_{\text{occ}}[1] + E_{\text{préc}}[0] + E_{\text{signe}}[3] + W_{\text{inc5}} + b_{\text{in}}), \quad (35)$$

une somme de 11 vecteurs appris de dimension 96, plus la projection des canaux continus. Deux odd lots placés 1 tick derrière l'ask partagent ainsi $E_{\text{pos}}[3] + E_{\text{taille}}[2]$, quel que soit le titre. Ce sont les couches suivantes (convolutions, attention) qui apprennent que leur *fréquence* et leur *enchaînement* diffèrent d'un titre à l'autre.

7.7 Étape 5 : les statistiques de la fenêtre

Sur la fenêtre de 100 événements (le motif répété), les blocs de la section 8 valent :

Bloc	Colonne	Valeur
ticks	spread_1	0,45
ticks	spread_2	0,5
ticks	spread_3_4	0,05
ticks	mid_move	0,39
ticks	move_half_tick	1
lots	flux_round	0,8
lots	flux_odd	0,1
lots	flux_mixed	0,1
lots	flux_exactly_1lot	0,45
lots	flux_ge_10lots	0,05
position	pos_inside	0,2
position	pos_best	0,5
position	pos_d1	0,05
position	pos_d2	0,15
position	pos_d6p	0,1
orders	orders_per_event	0,5
orders	repeat_share	0,5
orders	first_seen_U	0,1
orders	A_to_D	0,3
orders	D_trade_share	0,571
orders	cross_venue_repeat	0,4

TABLE 7 – Extrait des blocs de fenêtre, calculés par `features/window.py`.

Chaque valeur se vérifie sur le motif :

- `flux_round` = 0,8 : 16 événements sur 20 ont un flux multiple entier d'un lot. Les exceptions sont $t = 5, 12$ (odd) et $t = 11, 13$ (mixtes).
- `pos_inside` = 0,2 : exactement les 4 suppressions qui vident le meilleur niveau (section 7.5).
- `first_seen_U` = 0,1 : 1 ordre sur les 10 du motif (5012) apparaît d'abord en mise à jour.
- `A_to_D` = 0,3 : parmi les 10 répétitions, 3 passent d'un ajout à une suppression ($t = 4, 10, 13$).
- `D_trade_share` = 4/7 : 4 des 7 suppressions sont des exécutions ($t = 4, 7, 13, 18$).
- `cross_venue_repeat` = 0,4 : 4 des 10 répétitions changent de venue ($t = 4, 12, 13, 18$).
- `move_half_tick` = 1 : tous les mouvements du mid font exactement un demi-tick.

7.8 Pourquoi pas une normalisation relative : un contre-exemple chiffré

Deux fenêtres identiques en tout, sauf le spread : constant à 1 tick dans l'une, à 5 ticks dans l'autre. Ce sont typiquement un titre à gros tick et un titre à petit tick.

Représentation	Colonne	spread 1 tick	spread 5 ticks
avant (relative)	<code>spread_rel_mean</code>	0,693	0,693
avant (relative)	<code>spread_rel_std</code>	0	0
avant (relative)	<code>price_pos_rel_mean</code>	0	0
maintenant (ticks)	<code>spread_1</code>	1	0
maintenant (ticks)	<code>spread_5p</code>	0	1
maintenant (ticks)	<code>spread_t_median</code>	1	5

TABLE 8 – L'ancienne représentation (relative à la médiane locale) ne distingue pas les deux fenêtres (Proposition 5.2). Les colonnes en ticks les séparent sans ambiguïté.

8 Blocs de features de fenêtre

8.1 Le registre

Une feature de fenêtre est une fonction $\Phi : W \mapsto \mathbb{R}^d$. Elle est regroupée en *blocs* : un bloc correspond à une hypothèse. Un bloc s'écrit comme une fonction Python décorée :

```
@block('ticks', family='ticks') def ticks(r): return {nom: tableau (n,)}
```

Le registre calcule chaque bloc une fois par partition, par morceaux de 20 000 fenêtres, et l'écrit sur disque sous une clé

$$\text{clé} = \text{SHA256}(\text{source de la fonction} \parallel \text{source des helpers} \parallel (\hat{\delta}, \lambda, \# \text{venues}) \parallel \text{empreinte du cache brut})_{[10]}. \quad (36)$$

Modifier une fonction change sa clé : seul ce bloc est recalculé, et l'ancienne version est supprimée. Deux règles sont imposées par conception :

1. un bloc ne lit jamais les labels ;
2. un bloc ne calcule aucune statistique *entre* fenêtres (pas de quantile global). Tout ce qui s'ajuste sur une population (standardisation, seuils) relève des modèles et se fait sur le fit seulement.

8.2 Opérateurs

Pour un masque d'événements M et un masque de validité V :

$$\text{part}(M \mid V) = \frac{\sum_t M_t V_t}{\sum_t V_t}, \quad \text{part}(M \mid V) = \text{NaN} \text{ si } \sum_t V_t = 0. \quad (37)$$

$q_\theta(x)$ désigne le quantile θ de la fenêtre sur les valeurs finies, par interpolation linéaire. Il est calculé par un tri vectorisé, équivalent exact de `np.nanquantile` et vingt fois plus rapide.

8.3 Les onze blocs

Bloc	d	Définition
<code>base_relative</code>	18	moyenne et écart-type sur t de : $\log(1 + S_t/s^*)$, $\text{asinh}(h_t/(2s^*))$, $\text{asinh}((p_t - m_t)/(\delta s^*))$, I_t , $\log(1 + Q_t/d^*)$, $\text{asinh}(\Delta Q_t/d^*)$, $\text{slog}(f_t/f^*)$, $\mathbf{1}[n_t > 0]$, $\mathbf{1}[\rho_t = \rho_{t-1}]$. <i>Approximation des résumés du pipeline précédent : c'est la base du screening.</i>
<code>cat_freq</code>	12	parts de chaque venue (inconnue comprise), de chaque action, du côté B, des trades.
<code>cat_cross</code>	30	parts $\text{venue} \times \text{action}$, $\text{venue} \times \text{trade}$, $\text{action} \times \text{côté} \times \text{trade}$.
<code>levels</code>	7	$q_{1,.5,.9} \log(1 + Q/\lambda)$, $q_{.5} \log(1 + q^b/\lambda)$, $q_{.5} \log(1 + q^a/\lambda)$, $q_{.5}$ et moyenne de $\log(1 + F)$. <i>Niveaux absolus.</i>
<code>ticks</code>	14	$\pi_0, \pi_1, \pi_2, \pi_{3-4}, \pi_{\geq 5}$; $q_{.5} S$; moyenne de $\log(1 + S)$; part de mouvements du mid ($ h > .25$) ; parmi eux, parts d'un demi-tick, d'un tick, de plus d'un tick ; amplitude du mid en ticks ; part de prix hors grille ; part de retournements de signe entre mouvements consécutifs.
<code>lots</code>	9	parts de flux nul, rond, odd, mixte, exactement 1 lot, ≥ 10 lots ; part de carnets aux deux tailles rondes ; part de taille nulle à une cotation valide ; part de tailles négatives.
<code>position</code>	15	parts de D_t dans le spread, au meilleur, 1, 2, 3–5, > 5 , inconnu ; pour $a \in \{A, D, U\}$: part au meilleur, part plus profond ; part des trades au meilleur ; moyenne de $\log(1 + \max(D, 0))$.

Bloc	d	Définition
<code>orders</code>	22	ordres distincts / 100, part de répétitions, part d'ordres à plusieurs événements, $\max N_t/100$, même ordre que l'événement précédent, moyenne de $\log(1 + g)$, répétitions sur une autre venue, parts de première apparition A/D/U, matrice 3×3 des transitions $a_t^{\leftarrow} \rightarrow a_t$, part de trades parmi D et parmi U, « ajouté puis supprimé » observé.
<code>book_dyn</code>	10	moyenne, moyenne absolue et écart-type de I ; parts de changement du bid, de l'ask ; persistance du signe du flux ; changements de côté ; hausses et baisses de profondeur ; part de flux positifs.
<code>tokens_bow</code>	24	sac de mots : parts de action $\times$ côté $\times$ zone (dans le spread, au meilleur, proche $\leq 2,5$, loin).
<code>transitions</code>	36	bigrammes : parts des couples $(\text{action}, \text{côté})_{t-1} \rightarrow (\text{action}, \text{côté})_t$.

TABLE 9 – Les blocs de fenêtre (197 colonnes au total pour 6 venues).

Remarque 8.1 (Familles). Chaque bloc porte une *famille* : relative, category, level, ticks, lots, position, orders, dynamics, tokens. La famille `level` sert aux augmentations (section 12.4) : seules les colonnes de niveau reçoivent du bruit.

8.4 Invariances vérifiées

Proposition 8.2. *Multiplier toutes les quantités (q^b, q^a, f) d'une fenêtre par $c > 0$ laisse `base_relative` inchangé, et modifie `levels`.*

Démonstration. Les grandeurs de `base_relative` sont des rapports à des médianes de la même fenêtre (Proposition 5.2), ou bien l'imbalance, homogène de degré 0. Les quantiles de $\log(1 + cQ/\lambda)$ dépendent de c . $\square$

Les deux propriétés sont testées numériquement (fichier `tests/test_lab.py`).

9 Protocole : partitions, dépendance intra-journalière, audit

9.1 Partitions

Pour chaque classe c , avec $n_c = 6\,700$ fenêtres (`cfm/split.py`) :

1. **Audit** : tirer $\lfloor 0,10 n_c \rfloor$ fenêtres au hasard, avec une graine dédiée (`seed+10\,000`). Le tirage est donc identique quel que soit le réglage du stress.
2. **Stress** : parmi le reste, trier par $q_{1,5} \log(1 + Q/\lambda)$ (bloc `levels`) et prendre les $\lfloor 0,10 n_c \rfloor$ *plus faibles* si $\text{med}_{\text{test}} < \text{med}_{\text{train}}$ (mode `auto`), sinon les plus élevées.
3. **Valid** : $\lfloor 0,15 n_c \rfloor$ au hasard parmi le reste.
4. **Fit** : tout le reste.

Algorithme 9.1 – Construction des partitions

Tailles obtenues : fit 104 520, valid 24 120, stress 16 080, audit 16 080. Médianes de profondeur : 1,841 (train) contre 1,697 (test) : la queue *basse* est retenue. Le pipeline précédent codait `STRESS_TAIL='high'`. Son stress mesurait donc la robustesse à des carnets *plus* profonds, alors que le test l'est *moins*.

Remarque 9.1 (Transductivité). Le choix du sens utilise une statistique des entrées test (non étiquetées). C'est consigné dans `split.json` (`transductive: true`). Le mode `low` ou `high` rend le protocole purement « train only ».

9.2 Pourquoi la validation aléatoire surestime le test

Reprenons le modèle hiérarchique (3). Avec un split aléatoire au niveau des fenêtres, une fenêtre de validation du jour d a, en moyenne,

$$20 \times \frac{|\text{fit}|}{N_{\text{tr}}} = 20 \times 0,65 = 13 \quad (38)$$

« sœurs » du même titre et du même jour dans le fit. Le classifieur peut alors exploiter $\eta_{c,d}$: il reconnaît le *jour* autant que le titre. Formellement, notons A_{val} l'accuracy en validation aléatoire et A_{new} celle sur des jours jamais vus :

$$A_{\text{val}} = A_{\text{new}} + \underbrace{\Pr(\text{correct grâce à } \eta \mid \text{jour vu}) - \Pr(\text{correct grâce à } \eta \mid \text{jour non vu})}_{\Delta_\eta \geq 0 \text{ en général}}. \quad (39)$$

Δ_η grandit avec la variance de journée σ_η^2 et avec la capacité du modèle à mémoriser des régimes fins. Un réseau qui lit 100 événements en détail en a beaucoup plus qu'un MLP sur des moyennes. Le test ajoute un second écart : ses jours sont non seulement nouveaux, mais d'une *autre période* (covariate shift). Observation sur le run 1 :

$$A_{\text{val}} = 0,852 \quad \longrightarrow \quad A_{\text{LB}} = 0,570, \quad \text{écart} = 28 \text{ points}. \quad (40)$$

L'historique montrait déjà ce phénomène : la V3 rapportait 74,15 % en hold-out pour 50,24 % au leaderboard.

Conséquence sur les intervalles. Le bootstrap apparié de la section 10 rééchantillonne des fenêtres comme si elles étaient indépendantes. Avec une corrélation intra-jour ρ entre les indicatrices de réussite et $\bar{m}$ fenêtres par jour dans l'échantillon, la variance réelle d'une moyenne est multipliée par l'*effet de plan*

$$\text{deff} = 1 + (\bar{m} - 1)\rho. \quad (41)$$

Les intervalles rapportés sont donc trop étroits d'un facteur $\sqrt{\text{deff}}$, qui est inconnu faute de dates.

9.3 Ce que mesure chaque partition

Partition	Sert à	Limite
fit	apprendre ; scalers ; early stopping interne (tabulaire)	—
valid	choisir l'époque	partage les jours du fit : optimiste
stress	comparer les configurations (après équilibrage)	partage aussi des jours ; décalé en profondeur seule
audit	score final unique, décisions gelées	n'a jamais été ouvert
LB public	seule mesure hors période	coûteuse : à consulter avec parcimonie

Le fichier `LEADERBOARD.md` consigne chaque soumission avec la question qu'elle tranchait.

10 L'entonnoir de screening

Tester une feature en réentraînant un réseau coûte environ 10 minutes de GPU. L'entonnoir (`cfm/screen.py`) commence par des tests en secondes et ne réserve le réentraînement qu'aux blocs qui ont passé les étapes précédentes.

10.1 Étape 1 : utilité contre dérive, colonne par colonne

Utilité : rapport de corrélation η^2 . Pour une colonne x sur le fit (sous-échantillon stratifié de 60 000 fenêtres), avec $\bar{x}$ la moyenne globale et $\bar{x}_c$ la moyenne de la classe c :

$$\eta^2(x) = \frac{\sum_c n_c (\bar{x}_c - \bar{x})^2}{\sum_i (x_i - \bar{x})^2} \in [0, 1]. \quad (42)$$

C'est la part de variance expliquée par le titre : 1 si chaque titre a une valeur propre, 0 si la colonne est indépendante du titre en moyenne.

Dérive : statistique de Kolmogorov–Smirnov. Entre le train F et le test G (jusqu'à 60 000 fenêtres chacun) :

$$\text{KS}(x) = \sup_u |F_x(u) - G_x(u)|, \quad \text{et} \quad \text{shift}_{\text{sd}} = \frac{\bar{x}_{\text{te}} - \bar{x}_{\text{tr}}}{\text{sd}_{\text{tr}}(x)}. \quad (43)$$

Redondance. $\max_j |\text{corr}(x, z_j)|$ sur les colonnes z_j de la base. Le tableau signale aussi la part de NaN et la part de la valeur modale (colonne quasi constante).

Lecture de la carte (KS, η^2) . En haut à gauche : utile et stable, on garde. En haut à droite : utile mais sensible au régime ; on garde et on rend robuste. En bas : inutile seul ; seul le réentraînement tranche.

10.2 Étape 2 : AUC adversariale par bloc

Pour un bloc $\mathcal{B}$, on tire 30 000 fenêtres du train (étiquette 0) et 30 000 du test (étiquette 1). On entraîne un XGBoost à 200 arbres, et l'on calcule l'AUC des prédictions hors pli (3 plis) :

$$\text{AUC}(\mathcal{B}) = \Pr(\hat{s}(x_{\text{te}}) > \hat{s}(x_{\text{tr}})). \quad (44)$$

0,5 : indiscernable ; 1 : parfaitement séparable. Une AUC élevée *localise* le décalage. Elle n'impose pas de supprimer le bloc : un bloc décalé peut rester l'information la plus utile.

10.3 Étape 3 : ajout réentraîné (forward)

Soit une base $\mathcal{B}_0$ (`base_relative` + `cat_freq`, l'approximation des résumés précédents). Pour chaque candidat $\mathcal{B}$, on entraîne *le même* modèle, avec les mêmes fenêtres d'entraînement et la même graine, sur $\mathcal{B}_0$ puis sur $\mathcal{B}_0 \cup \mathcal{B}$. On évalue ensuite sur les *mêmes* fenêtres de valid et de stress. Avec $C_i^0, C_i^1 \in \{0, 1\}$ les indicatrices de réussite sur la fenêtre i :

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (C_i^1 - C_i^0), \quad \hat{\Delta}^{*(b)} = \frac{1}{n} \sum_{i=1}^n (C_{j_i^{(b)}}^1 - C_{j_i^{(b)}}^0), \quad j^{(b)} \sim \mathcal{U}\{1..n\}^n, \quad (45)$$

et l'intervalle à 95 % est $[\hat{\Delta}_{(2,5\%)}^*, \hat{\Delta}_{(97,5\%)}^*]$ sur $B = 1\,000$ tirages. L'appariement retire la variance due à la difficulté propre de chaque fenêtre. L'intervalle reste optimiste à cause de la dépendance intra-jour (section 9.2).

Règle de rétention (écrite avant les résultats). Un bloc est gardé si $\hat{\Delta}_{(2,5\%)}^* > 0$ en validation pour toutes les graines *et* $\hat{\Delta}_{\text{stress}} \geq -0,01$.

Modèle de screening. XGBoost `multi:softprob`, profondeur 6, $\eta = 0,1$, sous-échantillonnage 0,8 des lignes et 0,6 des colonnes, jusqu'à 2 000 arbres. L'arrêt anticipé (40 tours) se fait sur une tranche *interne* de 10 % du fit, jamais sur la validation, qui reste un score propre. Pour la vitesse, le screening utilise 50 % du fit.

10.4 Étape 4 : retrait réentraîné (ablation)

Sur l'ensemble retenu $\mathcal{S}$, on compare $\mathcal{S}$ et $\mathcal{S} \setminus \{\mathcal{B}\}$, de la même façon appariée. Un retrait qui ne coûte rien signale un bloc redondant avec les autres. Un masquage à l'inférence, au contraire, ne serait qu'un test de *sensibilité* : il ne dit pas ce qu'un modèle entraîné sans le bloc aurait appris à la place.

10.5 Sélection gloutonne

greedy ajoute à chaque pas le meilleur candidat, tant que $\hat{\Delta} \geq 0,002$ et que la borne basse est positive. La validation devient alors un score de *sélection* : l'ensemble obtenu doit être reconfirmé sur le stress et sur une autre graine.

11 SignatureNet

Fichier `cfm/neural/models.py`. Dimensions de la configuration de base : $d = 96$, $H = 4$ têtes, 2 blocs convolutifs, 2 blocs d'attention, contexte $D_c = 79$, $d_c = 64$, dropout $p = 0,15$.

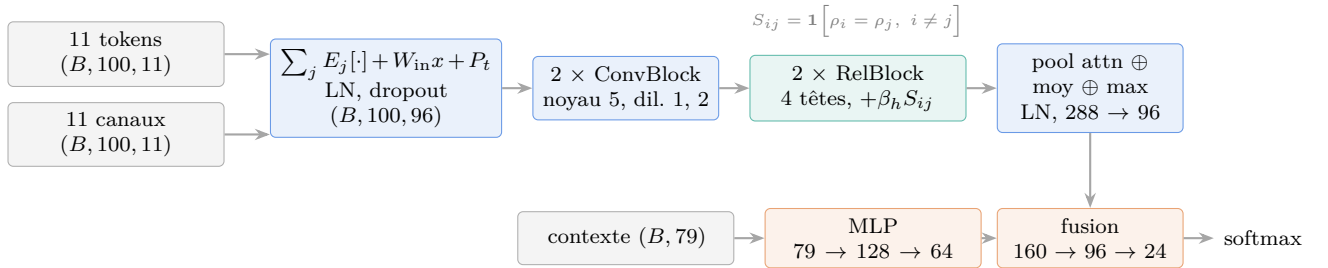


FIGURE 5 – Architecture de SignatureNet. En vert : le seul composant relationnel (le biais « même ordre »).

11.1 Encodage des événements

Soient $T = 11$ tokens $z_{t,1}, \dots, z_{t,T}$ (cardinalités C_j), $c_t \in \mathbb{R}^{11}$ les canaux continus standardisés et $P \in \mathbb{R}^{L \times d}$ une position apprise. L'entrée de l'encodeur est

$$x_t^{(0)} = \text{Dropout} \left(\text{LN} \left(P_t + \sum_{j=1}^T E_j[z_{t,j}] + W_{\text{in}} c_t + b_{\text{in}} \right) \right) \in \mathbb{R}^d, \quad E_j \in \mathbb{R}^{C_j \times d}. \quad (46)$$

La somme plutôt que la concaténation reprend le choix des Transformers de langue (mot + position + segment). Elle coûte $d \sum_j C_j = 96 \times 66$ paramètres. Les interactions entre tokens sont apprises plus loin, par les non-linéarités.

Standardisation. Pour chaque canal k , μ_k et σ_k sont estimés sur les événements des fenêtres du fit uniquement (refit : tous les labels). On applique $c_{t,k} \leftarrow \text{clip}((c_{t,k} - \mu_k)/\sigma_k, -8, 8)$, et $\text{NaN} \rightarrow 0$.

11.2 Blocs convolutifs dilatés

Pour $\ell = 0, 1$, avec une dilatation $\delta_\ell = 2^\ell$ et un noyau de taille 5 :

$$u = \text{Conv1d}_{5, \delta_\ell}(\text{LN}(x^{(\ell)})), \quad [\text{Conv1d}_{5, \delta}(y)]_t = \sum_{k=-2}^2 K_k y_{t+k\delta} + b, \quad (47)$$

$$x^{(\ell+1)} = x^{(\ell)} + \text{Dropout}(W_\ell \text{GELU}(u) + b_\ell), \quad (48)$$

avec un remplissage nul de 2δ de chaque côté (longueur conservée).

Proposition 11.1 (Champ réceptif). *Après les deux blocs, la sortie à la position t dépend des entrées $t - 6, \dots, t + 6$: 13 événements.*

Démonstration. Un noyau de taille 5 et de dilatation δ élargit le champ de 4δ . Le champ vaut donc $1 + 4 \cdot 1 + 4 \cdot 2 = 13$. Les connexions résiduelles et les couches linéaires par position ne l'élargissent pas. $\square$

11.3 Attention relationnelle « même ordre »

Soit $S \in \{0, 1\}^{L \times L}$, $S_{ij} = \mathbf{1}[\rho_i = \rho_j] \cdot \mathbf{1}[i \neq j]$: 1 si deux événements *distincts* concernent le même ordre. Pour un bloc et une tête h , avec $d_h = d/H = 24$:

$$[Q_h, K_h, V_h] = \text{LN}(x) W_h^{qkv}, \quad (49)$$

$$A_h = \text{softmax}_j \left(\frac{Q_h K_h^\top}{\sqrt{d_h}} + \beta_h S \right), \quad (50)$$

$$x \leftarrow x + \text{Dropout} \left([A_1 V_1 \parallel \dots \parallel A_H V_H] W^o \right), \quad (51)$$

$$x \leftarrow x + \text{Dropout} \left(W_2 \text{Dropout} \left(\text{GELU}(W_1 \text{LN}(x)) \right) \right), \quad W_1 \in \mathbb{R}^{d \times 2d}. \quad (52)$$

$\beta_h \in \mathbb{R}$ est *appris*, et initialisé à 0.

- À l'initialisation, (50) est l'attention standard.
- Si suivre les ordres aide la perte, le gradient

$$\frac{\partial \mathcal{L}}{\partial \beta_h} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial \text{logit}_{h,ij}} S_{ij} \quad (53)$$

pousse β_h vers des valeurs positives : l'événement i porte alors plus d'attention aux autres événements de son ordre.

— $\beta_h < 0$ correspond au cas inverse : la tête apprend à regarder *ailleurs* que l'ordre courant. Le biais n'utilise que S , donc que des égalités d'identifiants (Proposition 4.2). C'est une forme particulière de biais relationnel appris, dans l'esprit des représentations relatives de [6]. La variante `signature_no_bias` retire β . Au run 1, cette suppression ne change rien de mesurable (section 15).

11.4 Pooling

Avec $x \in \mathbb{R}^{L \times d}$ en sortie d'encodeur :

$$\omega_t = \text{softmax}_t(w_p^\top x_t + b_p), \quad \text{pool}_{\text{attn}} = \sum_t \omega_t x_t, \quad (54)$$

$$z_{\text{seq}} = \text{GELU} \left(W_s \text{LN} [\text{pool}_{\text{attn}} \parallel \frac{1}{L} \sum_t x_t \parallel \max_t x_t] + b_s \right) \in \mathbb{R}^d. \quad (55)$$

La moyenne mesure la *fréquence* d'un motif, le maximum sa *plus forte occurrence*, et le pooling attentionnel les événements jugés *importants*.

11.5 Contexte et fusion

$$z_{\text{ctx}} = \text{GELU} (W_{c2} \text{Dropout}(\text{GELU}(W_{c1} u + b_{c1})) + b_{c2}) \in \mathbb{R}^{64}, \quad u \in \mathbb{R}^{79} \text{ (blocs standardisés)}, \quad (56)$$

$$\text{logits} = W_{h2} \text{Dropout} (\text{GELU}(W_{h1} [z_{\text{seq}} \parallel z_{\text{ctx}}] + b_{h1})) + b_{h2} \in \mathbb{R}^{24}. \quad (57)$$

Le contexte apporte des statistiques globales, faciles à calculer mais difficiles à reconstruire par lecture séquentielle, comme « 85 % du temps à 1 tick ». Les blocs utilisés sont `levels`, `ticks`, `lots`, `position`, `orders` et `cat_freq`, soit $7 + 14 + 9 + 15 + 22 + 12 = 79$ colonnes.

11.6 Encodeurs de contrôle

Le BiGRU (`control_gru`, `gru_wide`) remplace convolutions et attention par

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (58)$$

$$\tilde{h}_t = \tanh(W_n x_t + b_n + r_t \odot (U_n h_{t-1} + b'_n)), \quad h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}, \quad (59)$$

dans les deux sens, avec $d/2$ unités par sens. `control_gru` a de plus une tête linéaire $[z_{\text{seq}} \| z_{\text{ctx}}] \rightarrow 24$.

11.7 Décompte exact des paramètres

Module	Paramètres	Calcul
Position P	9 600	100×96
Embeddings	6 336	66×96
Projection continue	1 152	$11 \times 96 + 96$
LN d'entrée	192	2×96
2 ConvBlocks	111 360	$2 \times (192 + 96 \cdot 96 \cdot 5 + 96 + 96 \cdot 96 + 96)$
2 RelBlocks	149 576	$2 \times (384 + 27\,936 + 9\,312 + 37\,152 + 4)$
Pooling attentionnel	97	$96 + 1$
Projection de pooling	28 320	$576 + 288 \cdot 96 + 96$
Contexte	18 496	$(79 \cdot 128 + 128) + (128 \cdot 64 + 64)$
Tête	17 784	$(160 \cdot 96 + 96) + (96 \cdot 24 + 24)$
Total signature	342 913	no_bias : $-8 (\beta)$
<code>control_gru</code>	110 105	GRU $2 \times 3 \times (48 \cdot 96 + 48 \cdot 48 + 2 \cdot 48) = 42\,048$
<code>gru_wide</code> ($d = 176$)	331 625	contrôle à capacité égale

TABLE 10 – Paramètres par module. Les totaux coïncident avec ceux mesurés dans le run Kaggle.

12 Entraînement, augmentations, refit

12.1 Perte

Avec un lissage d'étiquettes $\epsilon = 0,02$ et $\hat{p} = \text{softmax}(\text{logits})$:

$$\mathcal{L} = -\frac{1}{|B|} \sum_{i \in B} \sum_{c=1}^K \left((1 - \epsilon) \mathbf{1}[c = y_i] + \frac{\epsilon}{K} \right) \log \hat{p}_{i,c}. \quad (60)$$

12.2 Optimiseur et calendrier

AdamW [7], avec un pas η_k , $\beta = (0,9; 0,999)$ et une décroissance de poids $\lambda = 0,01$ découplée :

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2, \quad (61)$$

$$\theta_k = \theta_{k-1} - \eta_k \left(\frac{m_k / (1 - \beta_1^k)}{\sqrt{v_k / (1 - \beta_2^k)} + \varepsilon} + \lambda \theta_{k-1} \right). \quad (62)$$

Le gradient est écrêté en norme : $g \leftarrow g \cdot \min(1, 1/\|g\|_2)$. Le pas suit un warmup linéaire sur k_w pas (une époque), puis un cosinus jusqu'à 5 % du pas initial :

$$\eta_k = \begin{cases} \eta_0 \frac{k+1}{k_w} & k < k_w, \\ \eta_0 \left(0,05 + 0,95 \cdot \frac{1}{2} (1 + \cos \pi \frac{k-k_w}{K_{\text{tot}}-k_w}) \right) & k \geq k_w, \end{cases} \quad \eta_0 = 10^{-3}, \quad K_{\text{tot}} = E \cdot \lceil n_{\text{fit}}/512 \rceil. \quad (63)$$

Avec $E = 40$ époques et des lots de 512 fenêtres, $K_{\text{tot}} = 40 \times 205 = 8\,200$ pas. En précision mixte (fp16 sur GPU), la perte est multipliée par un facteur dynamique avant rétropropagation, puis les gradients sont divisés par ce facteur.

12.3 Sélection de l'époque

À chaque époque, on évalue en mode `eval` :

- la validation, qui *sélectionne* : on garde l'époque qui maximise la précision, puis à égalité minimise la log-loss ;
- le stress, en diagnostic seulement ;
- 5 000 fenêtres fixes du fit, pour mesurer l'écart d'apprentissage *avec les mêmes poids* que la validation.

L'arrêt se fait à E époques ou après 8 époques sans amélioration.

12.4 Augmentations

Elles sont appliquées à l'entraînement seulement.

- **Venue dropout** (p_v) : $z_{t,\text{venue}} \leftarrow 0$ avec probabilité p_v , indépendamment par événement.
- **Dropout de blocs de contexte** (p_b) : pour chaque fenêtre et chaque bloc $\mathcal{B}$, $u_{\mathcal{B}} \leftarrow 0$ avec probabilité p_b .
- **Jitter de niveaux** (σ_ℓ) : $u_j \leftarrow u_j + \sigma_\ell \xi$ pour les colonnes de famille `level`, avec $\xi \sim \mathcal{N}(0, 1)$ commun à la fenêtre.
- **Mise à l'échelle de la profondeur** (σ_d , v0.2.0) : un facteur $\gamma = e^{\sigma_d \xi}$ par fenêtre multiplie les tailles affichées du carnet. Sur une grandeur standardisée $\tilde{x} = (\log(1 + q/\lambda) - \mu)/\sigma$, on applique

$$\tilde{x} \leftarrow \frac{\log(1 + \gamma(e^{\sigma \tilde{x} + \mu} - 1)) - \mu}{\sigma}, \quad (64)$$

aux canaux $\log(1 + q^b/\lambda)$ et $\log(1 + q^a/\lambda)$ et aux quantiles de profondeur du contexte. Le flux et les parts de lots ne changent pas : les ordres restent en lots.

Proposition 12.1. *La mise à l'échelle de la profondeur conserve, à chaque événement, le rapport q_t^b/q_t^a , et donc l'imbalance I_t .*

Démonstration. Le même γ multiplie q_t^b et q_t^a : $(\gamma q^b)/(\gamma q^a) = q^b/q^a$, et I est homogène de degré 0. Le test de contrat le vérifie numériquement. $\square$

Motivation. Au run 1, le test a des carnets moins profonds ($\text{shift}_{\text{sd}} \approx -0,37$ sur la profondeur médiane), plus d'odd lots (+0,45) et moins de lots ronds (-0,42). Une hypothèse unique explique les trois : des *prix plus élevés* dans la période test. À montant égal, on affiche alors moins d'actions, et les quantités inférieures à un lot deviennent plus fréquentes. L'augmentation enseigne au modèle qu'un même titre peut afficher une profondeur différente d'un facteur γ . C'est une hypothèse : nous n'observons pas le niveau de prix.

12.5 Refit sur tous les labels

Une fois les décisions gelées, on réentraîne sur les 160 800 fenêtres (audit compris). Ce refit ne produit aucun score indépendant. Deux budgets sont possibles :

epochs : $K_{\text{tot}} = E \lceil N/512 \rceil$, arrêt à e^* époques (plus d'updates qu'en développement), (65)

steps : $K_{\text{tot}} = E \lceil n_{\text{fit}}/512 \rceil$, arrêt à k^* pas (même nombre d'updates). (66)

Le run 1 utilise `epochs`. Exemple pour `signature_s0` : $e^* = 38$, soit 11 970 pas, contre 7 790 en développement.

12.6 Reproductibilité et reprise

Chaque époque écrit `latest.pt` : poids, état de l'optimiseur, facteur d'échelle AMP, historique, états des générateurs aléatoires. Ce fichier est signé par

$$\text{SHA256}(\text{config résolue} \parallel \text{split} \parallel \text{clés des blocs} \parallel \text{code} \parallel \text{mode}). \quad (67)$$

Une reprise avec une autre configuration, un autre split, d'autres features ou un autre code est refusée.

12.7 Modèles tabulaires

XGBoost (GPU), LightGBM et un MLP à deux couches cachées de 512 partagent la même interface. L'arrêt anticipé se fait sur une tranche interne du fit, et le refit garde un nombre d'itérations fixe, égal au meilleur du développement. Ils servent de référence, de membres d'ensemble potentiels et de modèle de screening.

13 Ensemble et correction de prior par Sinkhorn

13.1 Ensemble

Pour M modèles de probabilités $P^{(m)} \in \Delta_K^N$, on prend la moyenne arithmétique

$$\bar{P} = \frac{1}{M} \sum_{m=1}^M P^{(m)}. \quad (68)$$

Au run 1, les membres sont `signature` et `signature_robust`, chacun avec deux graines. La moyenne simple ne demande aucune sélection de poids sur la validation : celle-ci reste donc un score propre du blend. Deux diagnostics accompagnent l'ensemble :

$$\text{accord}(m, m') = \frac{1}{N} \sum_i \mathbf{1}[\arg \max P_i^{(m)} = \arg \max P_i^{(m')}] , \quad \text{oracle} = \frac{1}{N} \sum_i \mathbf{1}[\exists m : \arg \max P_i^{(m)} = y_i] . \quad (69)$$

Un accord faible et un oracle élevé indiquent des erreurs complémentaires, donc un ensemble utile.

13.2 Le symptôme

Sur le test, les 81 600 prédictions $\arg \max \bar{P}_i$ se répartissent entre 1 320 (titre 2) et 7 112 (titre 7). Un test équilibré en contiendrait $81\,600/24 = 3\,400$ par titre (figure 6). Les titres sur-prédits (7, 13, 19, 15) sont ceux dont la profondeur est la plus faible au train, et le test est moins profond. Le décalage de covariables se traduit donc en un *biais de prior des prédictions*.

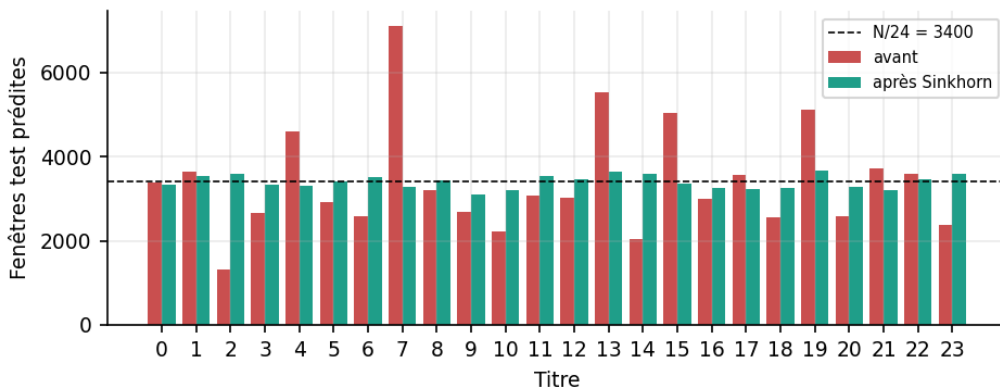


FIGURE 6 – Fenêtres test prédites par titre, avant et après équilibrage (blend des 4 finalistes, refit).

13.3 Rappel : correction de prior sous *label shift*

Supposons que seule la loi des classes change entre la source s et la cible t , et que $p_s(x | y) = p_t(x | y)$. Par la formule de Bayes,

$$p_t(y | x) = \frac{p(x | y)\pi_t(y)}{\sum_{y'} p(x | y')\pi_t(y')} = \frac{p_s(y | x) \frac{\pi_t(y)}{\pi_s(y)}}{\sum_{y'} p_s(y' | x) \frac{\pi_t(y')}{\pi_s(y')}}. \quad (70)$$

La correction optimale est donc un *coefficient par classe* $w_y = \pi_t(y)/\pi_s(y)$, suivi d'une renormalisation [1, 9]. Ici, π_s est uniforme (6 700 fenêtres par titre) et π_t l'est très probablement aussi. Le problème n'est donc pas un vrai label shift. Mais le covariate shift fait se comporter le modèle *comme si* son prior implicite avait changé. On cherche les coefficients qui ramènent la masse prédite au prior visé.

13.4 Formulation : projection entropique sur les marges

Soit $P \in \mathbb{R}_{>0}^{N \times K}$ (probabilités strictement positives, écrêtées à 10^{-12}), et $r = \mathbf{1}_N$, $c = \frac{N}{K} \mathbf{1}_K$ les marges visées. On cherche

$$Q^* = \arg \min_{Q \geq 0} \text{KL}(Q \| P) = \sum_{i,k} Q_{ik} \log \frac{Q_{ik}}{P_{ik}} - Q_{ik} + P_{ik} \quad \text{s.c.} \quad Q \mathbf{1}_K = r, \quad Q^\top \mathbf{1}_N = c. \quad (71)$$

Proposition 13.1 (Forme de la solution). *Il existe $u \in \mathbb{R}_{>0}^N$ et $w \in \mathbb{R}_{>0}^K$ tels que $Q_{ik}^* = u_i P_{ik} w_k$.*

Démonstration. Lagrangien : $\mathcal{L} = \text{KL}(Q \| P) + \sum_i a_i (\sum_k Q_{ik} - 1) + \sum_k b_k (\sum_i Q_{ik} - N/K)$. La condition de stationnarité donne $\partial \mathcal{L} / \partial Q_{ik} = \log(Q_{ik}/P_{ik}) + a_i + b_k = 0$, soit $Q_{ik} = P_{ik} e^{-a_i} e^{-b_k}$. On pose $u_i = e^{-a_i}$ et $w_k = e^{-b_k}$. Le problème est strictement convexe sur un polytope non vide (il contient $Q = \frac{1}{K} \mathbf{1} \mathbf{1}^\top$), donc la solution est unique. $\square$

Corollaire 13.2 (Règle de décision). $\arg \max_k Q_{ik}^* = \arg \max_k P_{ik} w_k$. *Le facteur de ligne u_i ne change pas la décision : l'équilibrage revient exactement à multiplier chaque titre par un coefficient w_k , comme dans (70).*

Proposition 13.3 (Qui bascule). *La fenêtre i , prédite c avant correction, est prédite $c' \neq c$ après si et seulement si*

$$\frac{P_{ic'}}{P_{ic}} > \frac{w_c}{w_{c'}} \quad \text{et} \quad c' = \arg \max_k P_{ik} w_k. \quad (72)$$

Une fenêtre confiante ($P_{ic} \gg P_{ic'}$) ne bascule que si le rapport des coefficients est extrême. Les fenêtres hésitantes basculent vers leur second choix. C'est ce qu'on observe (figure 8).

13.5 Algorithme de Sinkhorn–Knopp

-
1. $Q \leftarrow \max(P, 10^{-12})$.
 2. Répéter 50 fois :

colonnes : $Q_{ik} \leftarrow Q_{ik} \cdot \frac{N/K}{\sum_j Q_{jk}}$;
 lignes : $Q_{ik} \leftarrow Q_{ik} / \sum_{k'} Q_{ik'}$.
 3. Retourner Q .
-

Algorithme 13.1 – `sinkhorn_balance`

Chaque étape est une projection de Bregman (en divergence KL) sur l'un des deux ensembles de contraintes affines. Pour une matrice strictement positive, les itérations convergent vers l'unique Q^* de la Proposition 13.1 [3, 2]. Les produits cumulés des facteurs de colonnes donnent w , ceux des facteurs de lignes donnent u .

13.6 Exemple complet

Quatre fenêtres, deux titres, vérité (A, A, B, B) .

Fenêtre	Vrai	$P(A; B)$	avant	$Q^*(A; B)$	après
1	A	0,90 ; 0,10	A	0,838 ; 0,162	A
2	A	0,70 ; 0,30	A	0,573 ; 0,427	A
3	B	0,60 ; 0,40	A	0,463 ; 0,537	B
4	B	0,20 ; 0,80	B	0,126 ; 0,874	B
masse		2,40 ; 1,60	3/4	2,00 ; 2,00	4/4

Coefficients obtenus : $w_B/w_A = 1,739$. La fenêtre 3 bascule car $0,40/0,60 = 0,667 > 1/1,739 = 0,575$. La fenêtre 2 ne bascule pas : $0,30/0,70 = 0,43 < 0,575$.

13.7 Effet observé sur le test

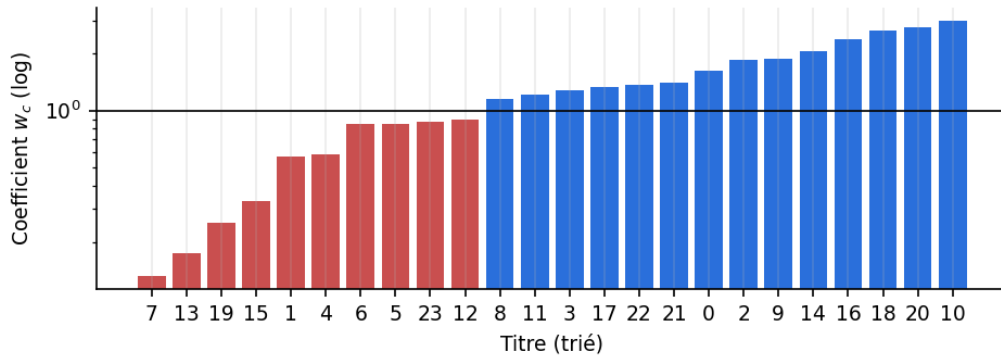


FIGURE 7 – Coefficients w_k (normalisés en moyenne géométrique). Titre 7 : $\times 0,13$; titre 10 : $\times 2,99$.

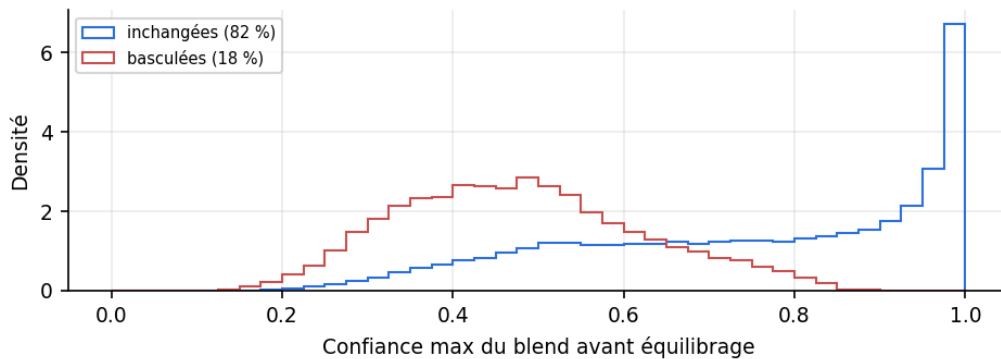


FIGURE 8 – 18 % des prédictions basculent. Leur confiance moyenne avant correction est de 0,48, contre 0,75 pour les fenêtres inchangées.

Blend des 4 finalistes	valid	stress	LB public
brut	0,8525	0,7411	0,5700
équilibré	0,8599	0,8055	0,5958
gain	+0,7 pt	+6,4 pts	+2,6 pts

TABLE 11 – L'équilibrage a été validé sur le stress *avant* d'être soumis. Le stress, lui-même décalé en profondeur, a correctement prédit le signe du gain au LB, mais pas son ampleur.

13.8 Conditions de validité

1. **Test équilibré.** Si π_t n'était pas uniforme, imposer $c = \frac{N}{K}\mathbf{1}$ dégraderait le score. Le gain observé au LB plaide pour l'uniformité, sans la démontrer.
2. **Bon classement relatif.** La correction ne change que l'importance relative des titres. Elle répare les fenêtres dont le bon titre est un second choix proche, pas les erreurs confiantes.
3. **Transductivité.** w dépend de *tout* le test. La prédiction d'une fenêtre dépend donc des autres fenêtres du test, et la conformité au règlement est à vérifier. Une variante non transductive consisterait à estimer w sur un ensemble de référence (le stress, par exemple). Elle serait moins adaptée au vrai décalage.
4. **Différence avec Saerens et al.** [1] *estiment* π_t par EM :

$$\pi_c^{(k+1)} = \frac{1}{N} \sum_i \hat{p}_{ic}^{(k)}, \quad \hat{p}_{ic}^{(k)} \propto P_{ic} \pi_c^{(k)} / \pi_s(c). \quad (73)$$

Nous *imposons* au contraire que la masse prédite soit uniforme. C'est une hypothèse plus forte, adaptée à un test construit avec 20 fenêtres par titre et par jour.

14 Round 2 : voisins et indicateurs sans labels

Ces outils ont été introduits en v0.2.0 (round 2), puis exécutés sur les vraies données en V3 : le vote entre voisins y fait passer le LB de 0,6028 à **0,6351** (section 15.8). Cette section décrit la méthode ; les résultats sont à la section 15.

14.1 Faire voter les fenêtres d'un même titre-jour

Le test contient environ 20 fenêtres par titre et par jour, et elles partagent l'effet $\eta_{c,d}$ de (3). Le mécanisme qui gonflait la validation aléatoire (reconnaître le jour) peut être retourné en notre faveur : si une représentation z rapproche les fenêtres d'un même titre-jour, une fenêtre ambiguë peut être corrigée par ses voisines plus nettes.

Grphe des k plus proches voisins. Pour un ensemble de fenêtres (le test entier, ou valid $\cup$ stress pour l'évaluation), on normalise $\hat{z}_i = z_i / \|z_i\|$, on note $\mathcal{N}_k(i)$ les k indices maximisant $\hat{z}_i^\top \hat{z}_j$ avec $j \neq i$, et l'on forme $A_{ij} = \frac{1}{k} \mathbf{1}[j \in \mathcal{N}_k(i)]$. Le calcul est exact, par blocs de 4 096 lignes, sur GPU si disponible.

Propagation. Pour $\alpha \in [0, 1)$:

$$Q^{(0)} = P, \quad Q^{(n+1)} = (1 - \alpha)P + \alpha A Q^{(n)}. \quad (74)$$

Proposition 14.1. Pour $\alpha < 1$, $Q^{(n)} \rightarrow Q^{(\infty)} = (1 - \alpha)(I - \alpha A)^{-1}P$.

Démonstration. A est stochastique par lignes, donc $\|A\|_\infty = 1$ et $\|\alpha A\|_\infty < 1$. L'itération est contractante, de point fixe unique $Q = (1 - \alpha)P + \alpha A Q$. La série de Neumann $\sum_n (\alpha A)^n$ converge vers $(I - \alpha A)^{-1}$. $\square$

C'est la propagation d'étiquettes de [8], appliquée à des probabilités *prédites* plutôt qu'à des labels connus. Le code utilise 1 ou 5 itérations, puis renormalise les lignes et applique Sinkhorn.

Trois représentations.

- **window** : blocs `levels`, `ticks`, `lots`, `cat_freq` et `position`, standardisés à l'intérieur de l'ensemble. Ce sont justement des statistiques de régime : leur dérive nuit à la classification, mais elles identifient bien le jour.
- **embedding** : $[z_{\text{seq}} \| z_{\text{ctx}}]$ des modèles, normalisé puis moyenné entre les membres. Les modèles de développement servent pour valid/stress, ceux du refit pour le test. Le code refuse d'utiliser les poids du refit sur des partitions étiquetées.
- **probs** : $\sqrt{P}$, soit une distance de Hellinger.

Pureté. Avant tout lissage, on mesure la part des voisins qui appartiennent au même titre :

$$\text{pureté@}k = \frac{1}{nk} \sum_i \sum_{j \in \mathcal{N}_k(i)} \mathbf{1}[y_j = y_i]. \quad (75)$$

Densité et facteur k . Le pool d'évaluation valid $\cup$ stress contient 25 % des fenêtres de chaque jour, soit environ 5 par titre-jour, alors que le test en contient 20. Le k choisi sur le pool est donc multiplié par 4 pour le test. C'est une *hypothèse*, écrite dans le code (`k_scale`).

Règle d'adoption. Le lissage n'est adopté que s'il améliore la précision *équilibrée* sur le pool d'au moins 0,5 point. Ce gain est un score de sélection, pas une estimation du gain sur le test.

14.2 Test préliminaire : l'espace des probabilités ne suffit pas

Sur les probabilités réelles du run 1, avec des voisins dans l'espace `probs` :

pool valid $\cup$ stress	brut	équilibré	meilleur lissage + équilibrage
précision	0,8079	0,8281	0,8281

C'était attendu : deux fenêtres voisines *en probabilités* portent la même information, donc la propagation n'ajoute rien. L'intérêt du lissage dépend d'une représentation qui encode le *jour*, ce que le round 2 mesure avec la pureté.

14.3 Indicateurs sans labels sur le test

Pour des probabilités test P (modèle de développement) :

$$\text{confiance} = \frac{1}{N} \sum_i \max_k P_{ik}, \quad \text{entropie} = -\frac{1}{N} \sum_{i,k} P_{ik} \log P_{ik}, \quad (76)$$

$$\text{KL d'équilibre} = \sum_k \hat{\pi}_k \log(K \hat{\pi}_k), \quad \hat{\pi}_k = \frac{1}{N} \sum_i \mathbf{1}[\arg \max P_i = k], \quad (77)$$

$$\text{accord entre graines} = \text{accord}(m, m') \text{ pour deux graines d'une même configuration.} \quad (78)$$

Au run 1, pour `signature_s0` :

	valid	test
confiance	0,858	0,738
accord entre graines	0,845	0,648
plus grand / plus petit nombre de prédictions par titre	≈ 1	5,4

Ces chiffres annonçaient, avant toute soumission, que le test serait bien plus difficile que la validation. Ils ne sont pas des accuracies. Ils ne servent à présélectionner des modèles qu'une fois *ancrés* à quelques scores LB connus.

15 Résultats du run 1 (Kaggle, GPU)

Les chiffres de cette section viennent des fichiers `lab_results.zip` du run 1 (6 octobre 2026). Le split est fixé : fit 104520, valid 24120, stress 16080, audit 16080 (jamais ouvert).

15.1 Screening univarié

Feature	η^2 (fit)	KS train/test	shift (sd)
levels:log_depth_q90	0,630	0,135	-0,28
levels:log_depth_q50	0,618	0,170	-0,37
ticks:spread_5p	0,563	0,024	-0,04
levels:log_depth_q10	0,510	0,174	-0,33
ticks:spread_1	0,504	0,018	+0,03
ticks:log_spread_t_mean	0,430	0,026	-0,06
levels:log_absflux_mean	0,415	0,172	-0,37
position:log_dist_mean	0,363	0,122	+0,31
lots:flux_odd	0,361	0,191	+0,45
lots:flux_round	0,353	0,183	-0,42

TABLE 12 – Les dix colonnes les plus séparatrices. En vert : les colonnes de tick, à la fois séparatrices et stables.

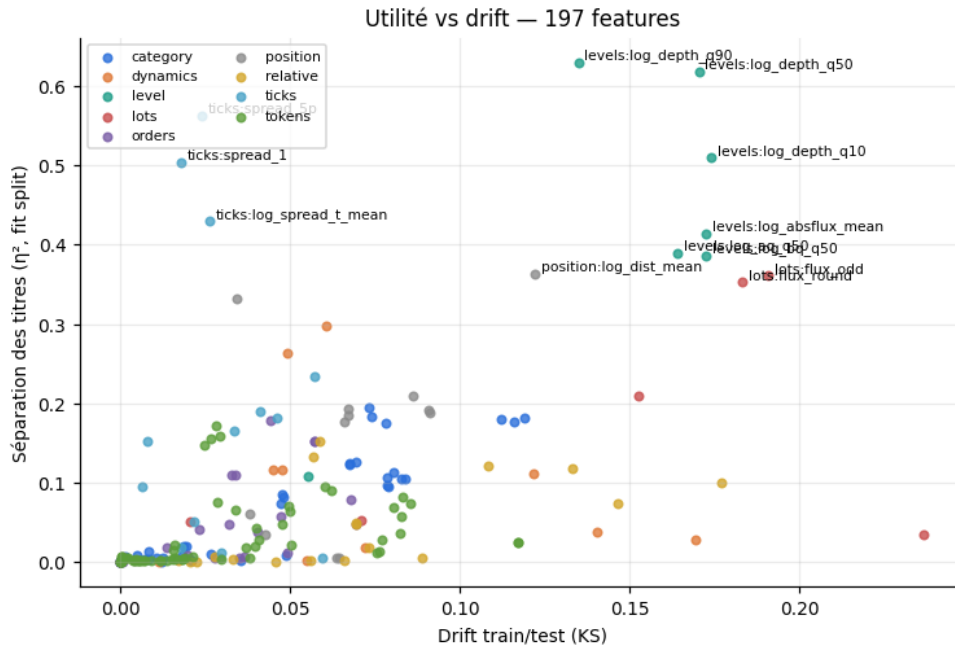


FIGURE 9 – Carte utilité/dérive des 197 colonnes.

Trois enseignements.

1. **Les ticks sont une signature stable.** $\eta^2 \approx 0,5$ avec $KS \approx 0,02$: c'est exactement ce que la section 5 prédisait de la forme discrète du spread.
2. **La profondeur est séparatrice et décalée.** Le test est moins profond de 0,3 à 0,4 écart-type.
3. **Les lots dérivent.** Le test contient davantage d'odd lots (+0,45 sd) et moins de lots ronds (-0,42 sd). Avec la baisse de profondeur, c'est cohérent avec l'hypothèse de prix plus élevés (section 12.4).

15.2 AUC adversariale

Bloc	d	AUC	Bloc	d	AUC
base_relative	18	0,733	book_dyn	10	0,669
levels	7	0,709	cat_cross	30	0,668
lots	9	0,699	cat_freq	12	0,668
position	15	0,670	transitions	36	0,653
tokens_bow	24	0,618	orders	22	0,596
ticks	14	0,568	tous	197	0,863

TABLE 13 – Séparabilité train/test, bloc par bloc. Le total (0,863) est cohérent avec l'ancienne AUC CatBoost (0,903). Aucun bloc seul n'explique le décalage : il est diffus.

Fait notable : la base relative du pipeline précédent est *la plus* décalée (0,733). Être invariant à l'échelle ne garantit pas d'être stable entre périodes.

15.3 Ajout réentraîné

Base base_relative+cat_freq : valid 0,313, stress 0,245 (XGBoost, 50 % du fit).

+ bloc	Δ valid	IC 95 %	Δ stress	retenu ?
position	+5,9	[+5,3 ; +6,4]	+3,8	oui
levels	+5,4	[+4,9 ; +5,9]	-13,3	non
lots	+5,2	[+4,7 ; +5,8]	+4,3	oui
ticks	+5,0	[+4,5 ; +5,5]	+5,3	oui
tokens_bow	+3,6	[+3,1 ; +4,2]	+2,1	oui
book_dyn	+3,5	[+3,0 ; +4,0]	-0,1	oui
transitions	+2,0	[+1,5 ; +2,5]	+1,7	oui
orders	+1,5	[+1,0 ; +1,9]	+0,8	oui
cat_cross	+0,6	[+0,2 ; +1,1]	+0,4	oui

TABLE 14 – En points d'accuracy. levels est le seul bloc écarté par la règle : il fait gagner 5,4 points en validation et en perdre 13,3 en stress. C'est un piège de régime.

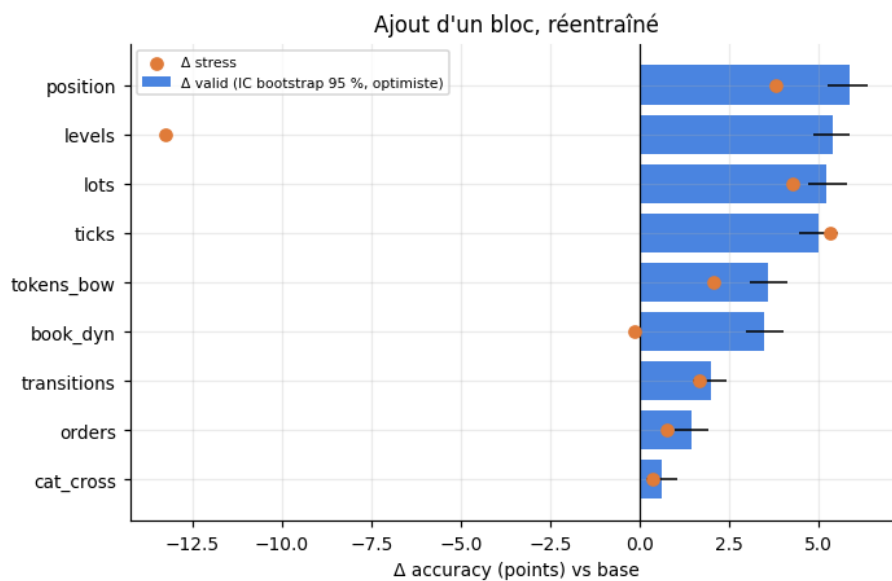


FIGURE 10 – Ajout réentraîné, valid et stress.

15.4 Retrait réentraîné

Sur les 10 blocs retenus (190 colonnes) : valid 0,465, stress 0,364.

– bloc	Δ valid	Δ stress	– bloc	Δ valid	Δ stress
lots	−4,1	−3,1	book_dyn	−0,7	−0,1
base_relative	−2,3	−0,05	transitions	−0,7	−0,1
ticks	−1,4	−1,6	tokens_bow	−0,4	0,0
orders	−1,2	−0,6	cat_freq	−0,2	−0,3
position	−0,9	−0,5	cat_cross	+0,1	−0,2

TABLE 15 – En points. `cat_cross`, `cat_freq` et `tokens_bow` sont redondants une fois les autres blocs présents.

15.5 Modèles

Run	Param.	Ép.	Min	Valid	Stress	Stress éq.
tab_lgbm	—	266 it.	3,7	0,480	0,372	—
tab_xgb	—	711 it.	1,4	0,494	0,384	—
tab_mlp	—	14	0,1	0,511	0,390	—
control_gru_s0	110 105	39	3,2	0,787	0,662	—
signature_relative_only_s0	340 289	38	10,4	0,766	0,669	—
signature_no_bias_s0	342 905	40	8,6	0,824	0,716	—
signature_s0	342 913	38	10,4	0,825	0,712	—
signature_s1	342 913	37	10,4	0,827	0,730	—
signature_robust_s0	342 913	39	10,5	0,830	0,689	—
signature_robust_s1	342 913	35	10,5	0,836	0,719	—
blend des 4 finalistes				0,852	0,741	0,806

TABLE 16 – Run 1. La colonne « stress éq. » (après Sinkhorn) n’a été calculée que pour le blend : elle devient le critère de sélection au round 2.

Lecture prudente.

- **Séquence contre tabulaire** : 0,79–0,84 contre 0,48–0,51 en validation. Lire les événements apporte beaucoup plus que des statistiques de fenêtre. Mais la validation aléatoire profite aussi davantage aux modèles séquentiels, qui reconnaissent mieux le jour (section 9.2).
- **signature contre control_gru** : +3,8 points en valid, +5,0 en stress. Mais le rapport de paramètres est de 3,1. Design et capacité sont confondus : c’est l’objet de `gru_wide` au round 2.
- **Biais « même ordre »** : 0,8251 avec, 0,8238 sans. L’écart est plus petit que celui entre graines (0,8251 et 0,8266). Pas d’effet mesurable : le retirer simplifie le modèle.
- **Robustesse** (venue dropout, jitter, dropout de blocs) : +0,7 point en valid en moyenne, −1,7 en stress. Pas de gain de transfert.
- **Niveaux** : les retirer coûte 5,9 points en valid et 4,3 en stress (une seule graine).
- **Maturité** : la meilleure époque tombe entre 35 et 40, mais c’est la fin du cosinus qui aplatit la courbe. L’écart en mode `eval` est de 0,91 (train) contre 0,82 (valid).

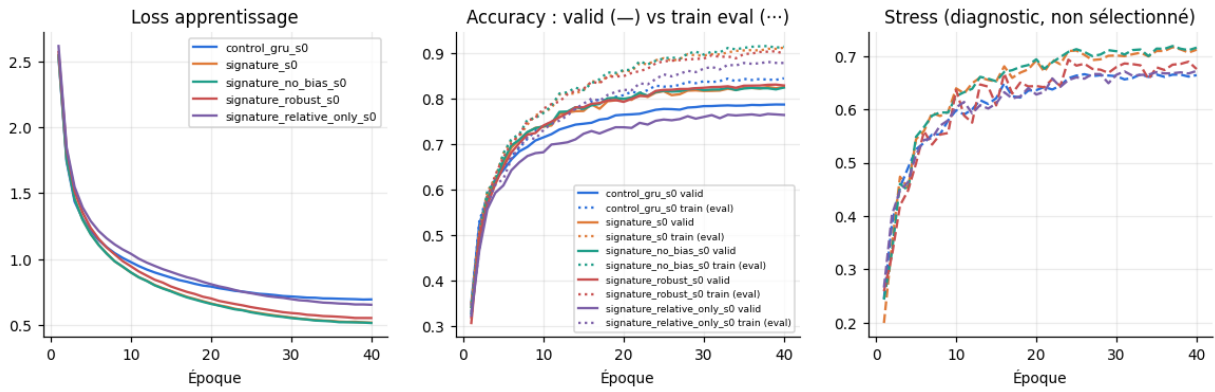


FIGURE 11 – Courbes d'apprentissage (seed 0). Le stress est tracé mais ne sélectionne rien.

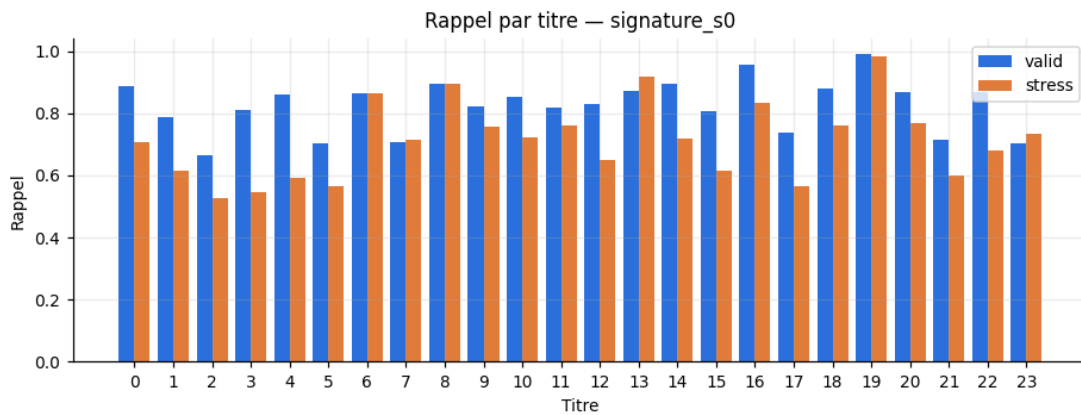


FIGURE 12 – Rappel par titre, signature_s0.

15.6 Du stress au leaderboard

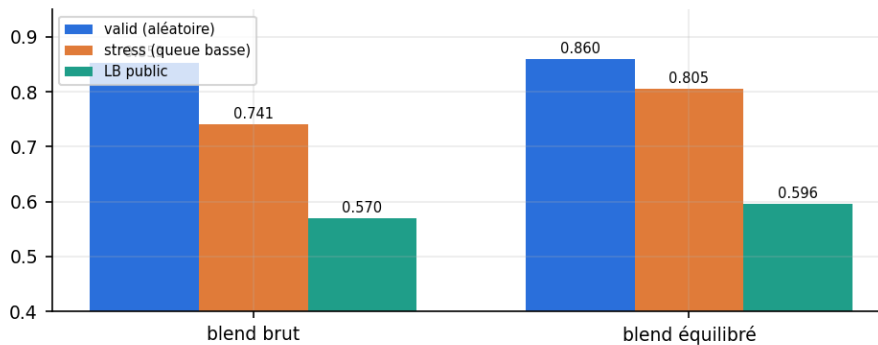


FIGURE 13 – Les trois mesures pour les deux soumissions. Le classement relatif (brut < équilibré) est le même partout. L'écart absolu, lui, ne l'est pas.

L'écart validation $\rightarrow$ LB est de 28 points. L'écart stress $\rightarrow$ LB est de 17 points (brut) et de 21 points (équilibré). Aucune partition interne n'estime donc le score public. En revanche, le stress a correctement *ordonné* les deux variantes. C'est pourquoi le round 2 classe les configurations sur le stress équilibré, et ancre les indicateurs sans labels sur les deux scores connus.

15.7 Round 2 : une différence à la fois

Six configurations, deux graines chacune, classées par précision *stress équilibrée*. Cette règle a été fixée avant les résultats, parce qu’au round 1 le stress avait correctement ordonné les deux soumissions.

Configuration (différence vs <code>signature_no_bias</code>)	Param.	Valid	Stress	Stress éq.	Δ
<code>sig2_depthaug</code> (profondeur $\times e^{\mathcal{N}(0;0,3)}$)	342 905	0,828	0,755	0,786	+1,3
<code>signature_no_bias</code> (référence)	342 905	0,828	0,725	0,773	—
<code>signature</code> (avec biais même ordre)	342 913	0,826	0,711	0,763	−1,0
<code>sig2_nolevels</code> (sans tailles absolues)	341 721	0,812	0,725	0,756	−1,7
<code>gru_wide</code> (BiGRU, capacité égale)	331 625	0,812	0,715	0,751	−2,2
<code>signature_robust</code>	342 913	0,832	0,689	0,750	−2,2

TABLE 17 – Round 2, moyenne de 2 graines. Les deux graines de `sig2_depthaug` donnent 0,7855 et 0,7858 en stress équilibré.

Enseignements :

- L’hypothèse « prix plus élevés » (section 12.4) tient en stress.
- À capacité égale, convolutions et attention battent le GRU : le gain du round 1 ne venait pas seulement de la taille.
- Les tailles absolues restent utiles au réseau, à l’inverse du screening tabulaire (section 10).

Soumission : `sig2_depthaug` $\times 2$ + `signature_no_bias` $\times 2$, équilibrée, donne **0,59999** (+0,4 point). Le stress annonçait +0,9 à +1,9 : il a prédit le sens de l’effet, mais en a surestimé l’ampleur.

15.8 V3 : validation par grappes, durée, alignement, voisins

Validation par grappes. Retirer de `valid` des grappes entières de régime fait baisser l’accuracy de l’ancre de 0,828 (aléatoire) à 0,772. C’est plus réaliste, mais encore loin du LB : la distance médiane de `valid` au fit (3,20) reste égale à celle de l’audit aléatoire (3,20). Les grappes de régime ne séparent qu’en partie les jours.

Configuration (différence vs <code>v3_base</code>)	Valid éq.	Stress éq.	Score	Δ
<code>v3_long</code> (60 époques)	0,789	0,793	0,791	+1,7
<code>v3_ema</code>	0,773	0,777	0,775	+0,1
<code>v3_depth05</code> ($\sigma = 0,5$)	0,774	0,775	0,775	+0,1
<code>v3_base</code> (<code>sig2_depthaug</code>)	0,772	0,776	0,774	—
<code>v3_nodepth</code>	0,774	0,763	0,768	−0,6
<code>v3_big</code> ($d = 128$, 681 k param.)	0,767	0,761	0,764	−1,0

TABLE 18 – V3, moyenne de 2 graines. Score = moyenne des deux précisions équilibrées. La meilleure époque de `v3_long` tombe à 58–59 sur 60 : l’entraînement n’est pas saturé. `v3_big` surapprend (0,977 sur le train contre 0,762 en validation).

Alignement de profondeur. Sur le stress, multiplier les tailles par $\gamma = e^{\Delta}$ (Δ = écart de log-profondeur médiane, estimé sans labels) fait passer la précision équilibrée de 0,8205 à 0,8268. Appliqué au test, $\gamma_{\text{test}} = 1,154$.

Représentation	Pureté@10	Précision éq. du pool après lissage
aucun lissage	—	0,8114
probs	0,755	0,8122
window (statistiques de régime)	0,297	0,8185
embedding	0,763	0,8215
combo (régime $\oplus$ embeddings)	0,756	0,8241

TABLE 19 – Pool = valid $\cup$ stress. Les statistiques de régime regroupent mal les titres (pureté 0,30), mais elles complètent les embeddings. Meilleur réglage : **combo**, $k = 10$, $\alpha = 0,5$, 5 itérations, appliqué au test avec $k = 40$.

Soumission V3	Contenu	LB public
A	$v3_long \times 3 + v3_ema \times 3$, $\gamma = 1,154$, Sinkhorn	0,6028
B	A + vote entre voisins (combo , $k = 40$)	0,6351

Vote entre voisins. C’est le deuxième plus gros gain du projet (+3,2 points), et le premier dont l’effet au LB *dépasse* l’effet mesuré en interne (+1,3 sur le pool). L’explication proposée : le test contient les 20 fenêtres de chaque titre-jour, contre environ 5 dans le pool. Chaque fenêtre test dispose donc de bien plus de « sœurs » pour voter.

16 Limites, menaces à la validité, suite

16.1 Ce qui n’est pas établi

1. **Attribution causale du gain.** Le passage de 0,5072 à 0,5700 mêle quatre changements : la représentation (unités, tokens, parcours d’ordres), l’architecture, l’ensemble de 4 modèles et le refit. Aucune soumission n’isole l’un d’eux. Le fait que **control_gru**, qui reçoit les nouvelles entrées, dépasse nettement le GRU de R1 en validation suggère que la représentation pèse lourd. Mais la validation est biaisée.
2. **Validation non représentative.** Faute de dates, aucune partition ne tient des jours entiers à l’écart. Toutes les mesures internes sont optimistes. Les intervalles bootstrap sous-estiment la variance d’un facteur $\sqrt{\text{deff}}$ inconnu.
3. **Leaderboard public.** Il est mesuré sur une partie du test, inconnue ici. Le consulter trop souvent ferait sur-apprendre ce sous-ensemble : **LEADERBOARD.md** limite et documente les soumissions.
4. **Hypothèse du prix.** « Prix plus élevés dans la période test » relie trois observations (profondeur, odd lots, lots ronds), mais le niveau de prix n’est pas observé.
5. **Transductivité.** Le sens du stress, l’équilibrage et le lissage par voisins utilisent les entrées test. Il faut vérifier que le règlement l’autorise. Dans le cas contraire, la soumission de référence est 0,5700.

16.2 Prochaines expériences

Expérience	Hypothèse	Abandon si
V4 : pseudo-étiquetage	apprendre sur des fenêtres test étiquetées par V3 B (accord lissé/brut, quota par titre) adapte le modèle à la période	$C_k40 \leq D_k40$ au LB
V4 : 90 époques	l'entraînement n'était pas saturé à 60	gain inférieur à l'écart entre graines
V4 : k au test	$k = 40$ n'est pas optimal	variantes à $\pm 0,3$ pt de B
Token $D_t^{\text{pré}}$ (§7.5)	distinguer « où était l'ordre » de « ce qu'il a fait au carnet »	stress équilibré inchangé
SignatureLite	3 convolutions + pooling (≈ 100 k paramètres) suffisent, sans attention	écart $> 0,5$ pt vs v3_long
Regroupement explicite en jours	voter par paquet de 20 fenêtres plutôt que par k voisins	pas de gain sur B

16.3 Conclusion

Le gain le plus robuste de ce travail ne vient pas d'une couche de réseau. Il vient de trois décisions. D'abord, *regarder* la donnée dans ses propres unités, ce qui a fait apparaître le régime de tick comme signature stable. Ensuite, *refuser* de croire la validation aléatoire. Enfin, *diagnostiquer le test sans labels* avant de le soumettre. L'équilibrage de Sinkhorn n'est qu'une conséquence de ce diagnostic : une projection entropique de deux lignes, dont la justification tient en une formule de Bayes.

A Notation

Symbole	Signification
$K = 24, L = 100$	nombre de titres, événements par fenêtre
$\delta = 0,01, \lambda = 100$	tick, lot
$v_t, o_t, a_t, s_t, \tau_t$	venue, ordre, action, côté, trade
p_t, b_t, α_t	prix de l'ordre, meilleur bid, meilleur ask (recentrés)
q_t^b, q_t^a, Q_t, I_t	tailles aux meilleures limites, profondeur, imbalance
f_t, F_t	flux, flux en lots
S_t, h_t, D_t	spread en ticks, Δ mid en demi-ticks, distance au meilleur prix de son côté
$\rho(t), n_t, g_t, a_t^{\leftarrow}, a_t^{\rightarrow}, N_t$	rang d'ordre, occurrence, écart, action précédente, première action, taille du parcours
s^*, d^*, f^*	médianes positives locales (spread, profondeur, $ f $)
$\pi_k(W)$	part du temps à k ticks de spread
$d = 96, H = 4, d_h = 24$	largeur, têtes, largeur par tête
S_{ij}, β_h	indicatrice « même ordre », biais appris de la tête h
$P, \bar{P}, Q^*$	probabilités d'un modèle, du blend, après Sinkhorn
u_i, w_k	facteurs de ligne et de classe (Sinkhorn)
A, α, k	graphe des voisins, poids de propagation, nombre de voisins

B Hyperparamètres

Élément	Valeur (configs/base.json)
Split	audit 10 %, stress 10 % (queue <code>auto</code>), valid 15 %, seed 42
Screening	XGBoost, base <code>base_relative+cat_freq</code> , 50 % du fit, seed 0
SignatureNet	$d = 96$, $H = 4$, 2 conv, 2 attentions, contexte 64, dropout 0,15
Entraînement	lot de 512, AdamW 10^{-3} , wd 0,01, warmup 1 époque, cosinus, 40 époques, patience 8
Perte	entropie croisée, lissage 0,02, écrêtage du gradient 1
Augmentations (robust)	venue 0,1 ; blocs 0,1 ; jitter de niveaux 0,25
Augmentation (depthaug)	$\sigma_d = 0,3$
Refit	budget <code>epochs</code>
Sinkhorn	50 itérations, cible uniforme, écrêtage 10^{-12}
Voisins	$k \in \{3, 5, 10, 20\}$, $\alpha \in \{0,3; 0,5; 0,8\}$, 1 ou 5 itérations, $k_{\text{test}} = 4k$

C Contrats testés (tests/test_lab.py, 15 tests)

1. labels alignés par `obs_id` même si `y_train` est mélangé ;
2. fenêtres non contiguës refusées ;
3. rang d'ordre invariant par renumérotation bijective ; identifiants manquants jamais égaux ;
4. helpers de parcours d'ordre égaux à une boucle naïve ;
5. unités en ticks sur une fenêtre construite à la main ;
6. `base_relative` invariant à l'échelle des tailles, `levels` sensible ;
7. cache de blocs indexé par le contexte (δ, λ) ;
8. partitions disjointes et complètes, 24 classes partout, audit indépendant du réglage du stress ;
9. tabulaire de bout en bout, prédictions test alignées ($\text{accuracy} > \text{hasard}$ sur le synthétique) ;
10. réseau : reprise identique, configuration différente refusée, refit, cinq encodeurs ;
11. augmentation de profondeur, extraction de représentations, refus d'utiliser les poids du refit sur des partitions étiquetées ;
12. la mise à l'échelle ne touche que les canaux de profondeur et conserve q^b/q^a ;
13. le lissage par voisins corrige des erreurs isolées ; le voisinage exclut la fenêtre elle-même ;
14. les indicateurs sans labels sont nuls sur des prédictions équilibrées ;
15. Sinkhorn : lignes de somme 1, colonnes de somme N/K .

D Reproduire

```

pip install -r requirements.txt
python scripts/kaggle_upload.py --notes "..."          # code -> Dataset Kaggle privé
# Kaggle : CFM_Signature_Lab.ipynb (round 1) puis CFM_Round2.ipynb
python tests/test_lab.py                                # 15 contrats, CPU
python -m cfm neural --config configs/signature.json --name sig_s0 --refit epochs
python -m cfm blend --runs sig_s0 sig_s1 --submit --balance

```

Pour le run 1, les fichiers `runs/*/result.json`, `history.csv`, les probabilités `*.npz` et `ledger.csv` se trouvent dans `lab_results.zip`.

Références

- [1] M. Saerens, P. Latinne, C. Decaestecker. *Adjusting the outputs of a classifier to new a priori probabilities : a simple procedure*. Neural Computation, 14(1), 2002.

- [2] M. Cuturi. *Sinkhorn distances : lightspeed computation of optimal transport*. NeurIPS, 2013.
- [3] R. Sinkhorn, P. Knopp. *Concerning nonnegative matrices and doubly stochastic matrices*. Pacific Journal of Mathematics, 21(2), 1967.
- [4] R. Cont, A. Kukanov, S. Stoikov. *The price impact of order book events*. Journal of Financial Econometrics, 12(1), 2014. arXiv :1011.6402.
- [5] A. Vaswani et al. *Attention is all you need*. NeurIPS, 2017.
- [6] P. Shaw, J. Uszkoreit, A. Vaswani. *Self-attention with relative position representations*. NAACL, 2018.
- [7] I. Loshchilov, F. Hutter. *Decoupled weight decay regularization*. ICLR, 2019.
- [8] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, B. Schölkopf. *Learning with local and global consistency*. NIPS, 2003.
- [9] Z. Lipton, Y.-X. Wang, A. Smola. *Detecting and correcting for label shift with black box predictors*. ICML, 2018.